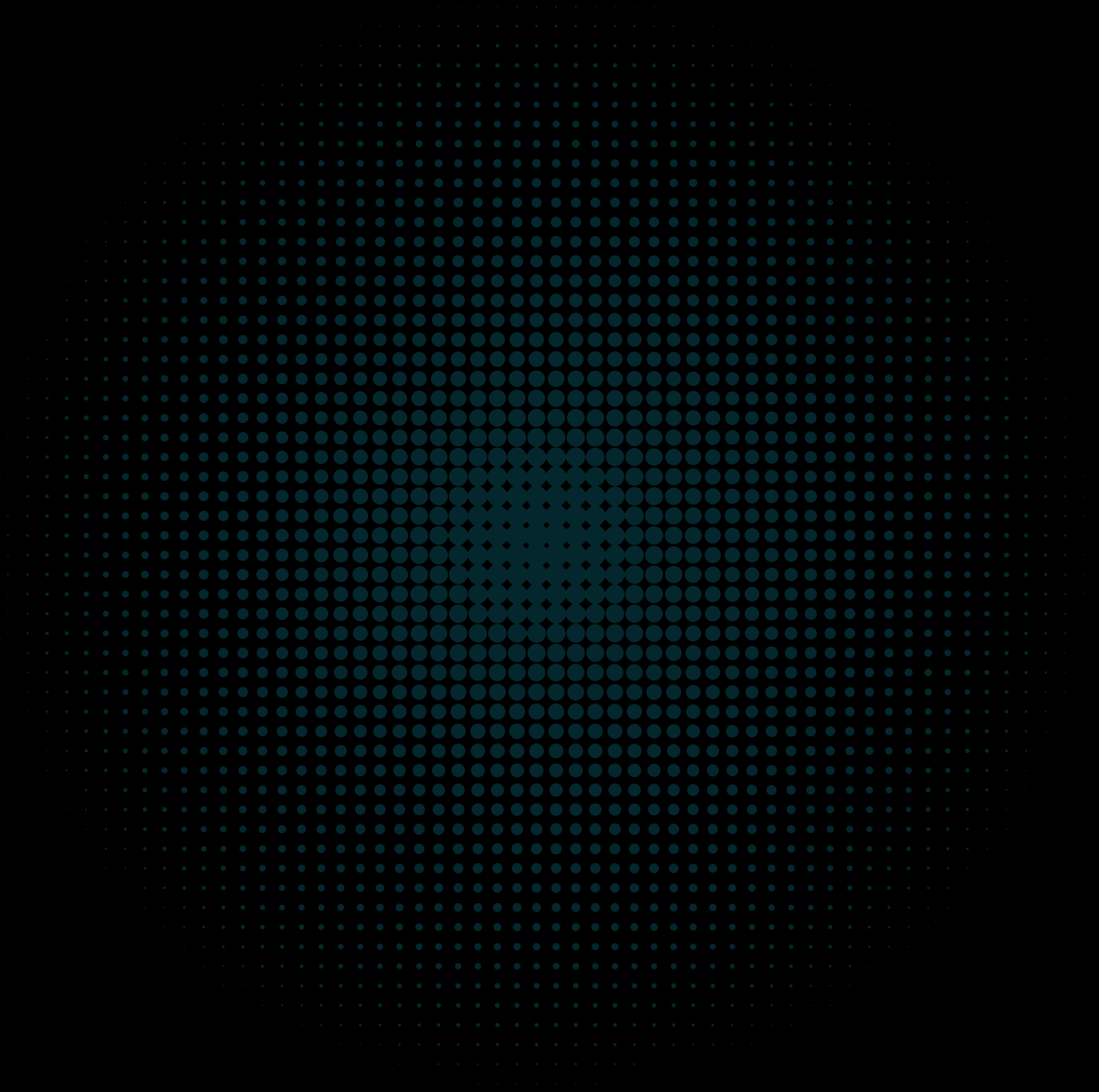


cmta.

CMTAT Aztec Specification

**Functional specifications of CMTAT
on Aztec (Ethereum L2)
to enable confidential balances and transfers.**

CMTAT Private Aztec version: v0.5.0
First published: September 2026



Private CMTAT security token — technical documentation

This is the full specification and design rationale. The repository [README](#) is the short version: what the project is, how to build and test it, and where to read next.

This project implements a private version of the CMTAT security token, using [Aztec](#). This allows banks and financial institutions to benefit from tokenization while maintaining privacy and compliance.

[Aztec](#) is a privacy-focused Layer 2 solution on Ethereum that enables confidential transactions using zero-knowledge proofs (ZKPs).

[CMTAT](#) is a framework for the tokenization of securities in compliance with local regulations. This project integrates Aztec with CMTAT, allowing financial institutions to adopt the standard while preserving transaction confidentiality.

This repository contains a functional private CMTAT prototype, where transactions remain private for users, while issuers retain the ability to audit and monitor activity to ensure compliance. Institutions can therefore participate in tokenized markets without publishing holdings or counterparties, which a public blockchain otherwise forces them to do.

Disclaimer: This project has not undergone an audit and is provided as-is without any warranties.

Table of contents

- [Key terms](#)
 - [How Aztec works, in short](#)
 - [Terms](#)
- [Deployment variants](#)
- [Functionalities overview](#)
- [Private token implementation](#)
 - [Assumptions and requirements](#)
 - [Storage](#)
 - [Overview](#)
 - [Mint private specifications](#)
 - [Transfer private specifications](#)
 - [The enqueued public half, `self.enqueue\_self\_transfer\(\)`](#)
 - [Burn private specifications](#)
 - [Batching limits](#)
 - [Events](#)
 - [Security and confidentiality properties](#)
 - [Modules](#)
 - [Delay of the delayed values](#)
 - [Issuer's view of transactions and notes](#)

- [Private/public bridges](#)
- [AIP-20 private profile](#)
- [Deployment](#)
- [Tests](#)
- [Gas sponsorship](#)
- [Comparison with Solidity CMTAT](#)
- [Comparison with CMTAT-Confidential \(Zama FHE\)](#)
- [Limitations](#)
- [Miscellaneous](#)
- [FAQ](#)
- [Glossary](#)
 - [Aztec protocol](#)
 - [Aztec.nr and the code in this repository](#)
 - [CMTAT and this project](#)
- [Intellectual property](#)
- [Security policy](#)

Key terms

How Aztec works, in short

Everything below depends on one property of the platform, so it is worth stating before the terms.

An Aztec contract has **two halves**. The **private** half runs on the user's own device and is proved there: it reads and writes encrypted *notes*, and what reaches the chain is commitments — hashes of data the network never sees. The **public** half runs on the sequencer, in the open, exactly like an EVM contract.

Three consequences shape this token:

- **A private function cannot read current public state.** It is proved against a historical snapshot, so anything it must check has to be published in a form whose value cannot change for a known window. That is what `DelayedPublicMutable` is, and the delay is its price.
- **Private state is append-only.** A note is never updated: it is spent by publishing its *nullifier*, and a new note is created. Only the owner can produce that nullifier, which is why the issuer can read a balance from its copies but cannot move it.
- **Private can call public, never the other way round.** A private function *enqueues* a public call that the sequencer runs afterwards, with no return value. A revert there reverts the whole transaction, including the private half.

The rest of the design follows from those three: what is public, what is delayed, and why a transfer's public half takes no arguments at all.

Terms

Enough to read the rest of this document. The full [Glossary](#) at the end defines every term used in the specification and the code.

Term	Definition
Private state	Contract state held as encrypted notes in each holder's own client, not in public storage. A balance is the sum of a holder's notes; only they can spend them.
Public state	Ordinary onchain state, readable by anyone. Here it carries the total supply, the pause and freeze flags, the transfer-restriction lists and the role table.
Note	The unit of private state — a small record holding an amount. Its <i>hash</i> is published onchain; its content is not.
Nullifier	The value published when a note is spent, derived from the note and its owner's key. Duplicates are rejected, which is what prevents double-spending — and why nobody but the owner can spend a note.
PXE	<i>Private eXecution Environment</i> — the client-side component holding a user's keys and notes, and running private functions. One PXE cannot read another's notes.
Issuer audit copy	Every note created is delivered twice: to its owner, and to the issuer. That second copy is what lets the issuer reconstruct balances for compliance.
Authwit	<i>Authentication witness</i> — a one-shot authorisation letting a third party perform one exact call on your behalf. The Aztec counterpart of an ERC-20 <code>approve</code> + <code>transferFrom</code> .
Delay on compliance flags	Freezing an address, changing a list entry or changing the issuer takes effect only after a delay, because private functions can only read public state that is guaranteed stable. See Limitations .

Deployment variants

Noir has no inheritance and allows one contract per package, so the variants are separate contract packages over a shared module library (`lib/`), built together as a Nargo workspace. A second library crate, `test-helpers/`, holds the helpers the three Noir suites share — advancing the chain past a delay, calling a private function on behalf of another account — so that they do not ship inside `lib/`.

Variant	Contents
<code>CMTATAztecLight</code>	Private token, pause, deactivation, freeze, access control, terms, version — no transfer restriction lists
<code>CMTATAztec</code>	The above plus the validation module (blacklist / whitelist)
<code>CMTATAztecDebt</code>	The above plus credit events and debt, for bond-like instruments

Two further contracts are not tokens but **ARC-403 authorization contracts**: they apply CMTAT's pause, deactivation, freeze and sender-side blacklist / whitelist to the stock tokens of the [CMTA fork of aztec-standards](#), which call them as a hook before every transfer and burn. See [doc/auth/README.md](#).

Contract	Restricts
<code>CMTATAztecAuth</code>	AIP-20 <code>Token</code>
<code>CMTATAztecAuthMultiToken</code>	ARC-1155 <code>MultiToken</code>

Because there is no inheritance, an entry point added to a shared module has to be declared in each variant's `main.nr` that should expose it.

Functionalities overview

The private CMTAT supports the following core features:

- **Private** mint, burn, and transfer operations
- **Public** pause of the contract, permanent deactivation, and public freeze of specific accounts
- **Auditability** of users private transactions by a central issuer
- **Transfer restriction** via address blacklisting/whitelisting

Unlike the reference [Solidity CMTAT](#), it does not support:

- Upgradeability
- An ERC-2771 meta-transaction module — unnecessary here, because Aztec sponsors gas natively; see [Gas sponsorship](#)

This reference implementation aims to fulfill the criteria required to tokenize financial instruments such as bonds, equity shares, and private credit notes.

You may modify the token code by adding, removing, or modifying features, at your own risk.

Private token implementation

Assumptions and requirements

- **Assumptions:**
 - **Total supply visibility:** The `totalSupply` should remain public and be updated according to mint and burn operations.
 - **Issuer and admin addresses:** The addresses of the issuer and admin can be publicly known.
 - **Third-party transactions:** We want to allow third parties to execute transactions on behalf of our users, so we use **authentication witnesses** when transferring. (same functionality as `transferFrom` on EVM)
 - **Mint and burn restrictions:** There is no authentication witness in the `mint_to_private` and `burn` functions, as a third party is not allowed to mint or burn; only the issuer can perform these actions.

- **Admin role:** The admin can add or remove other admins, with `grant_role` / `revoke_role` on `DEFAULT_ADMIN_ROLE`; an admin cannot revoke *itself*, and steps down with `renounce_role`. What cannot change is which role administers a role: `getRoleAdmin` returns `DEFAULT_ADMIN_ROLE` for every role, so no role can be delegated to another administrator. There is no two-step handover, so a grant to a mistyped address cannot be undone from that address's side.

- **The model is OpenZeppelin's `AccessControl` (v5)**, which CMTAT Solidity builds on, ported rather than reinvented. Three fingerprints of it survive in `access_controlModule.nr: getRoleAdmin(role)` keeps OpenZeppelin's camelCase name against this project's snake_case convention, `renounce_role` takes a `callerConfirmation` argument that must equal the caller — the guard OpenZeppelin 5.0 added to `renounceRole` — and the failed-authorisation message is its custom error name, `AccessControlUnauthorizedAccount`.
- **Two deliberate divergences.** OpenZeppelin exposes `_setRoleAdmin`, so a role there *can* be delegated to a different administrator; this module has no equivalent, which is why every role answers `DEFAULT_ADMIN_ROLE` and the hierarchy is fixed at one level. And `revoke_role` here refuses `sender == account` (Revoke Role: Cannot revoke role from self), where OpenZeppelin allows an admin to revoke its own role — self-removal must go through `renounce_role` and its confirmation.

- **Private by default, public by the holder's choice:** balances are private notes. If, and only if, the token was deployed with `public_side_enabled = true`, a holder may move value between its notes and a public balance through the four AIP-20 bridges; what such a move publishes is the mover's own side. See [Private/public bridges](#).

- **Functionalities:**

- **Totalsupply - Public Context:** For a particular CMTAT token, anyone may know the total number of tokens in circulation at any point in time.
- **BalanceOf - Private Context:** For a particular CMTAT token and a particular user, no one apart from the issuer should know the number of tokens currently recorded on the user's ledger address.
- **Transfer - Private Context:** Users may transfer some or all of their tokens to another ledger address (which the transferor does not necessarily control). Each transfer must remain private: only the transacting parties and the issuer may know that the transfer occurred, who the participants are, and how much was transferred.

Note: The issuer cannot do a force transfer on behalf of the user, as he would do in the Solidity version of CMTAT. The solution is that in the case where we want to have the same behaviour as a force transfer, we freeze the account.

- **Mint - Private Context** Issue a given number of tokens to a given ledger address. The issuer and the recipient should be the only ones who know that a transaction is happening. Only the issuer and the receiving address should know the amount minted.

Note: According to the assumption, the total supply will increase accordingly in a public function, and thus the new total supply will be visible to everyone. The supply change amount will be traceable to that particular private proof.

- **Burn - Private Context** The issuer burns (destroys) a given number of tokens from a given ledger address. The issuer and the given address should be the only ones who know that a transaction is happening.

Note: Under the above assumptions, a public function will reduce the total supply when a burn happens. Therefore, the updated total supply will be visible to everyone, and the amount of the change can be traced back to a specific private proof.

Overview

Three deployment variants compose modules from one shared library. Noir has no inheritance and allows one contract per package, so a variant is a different *composition*, not a subclass.

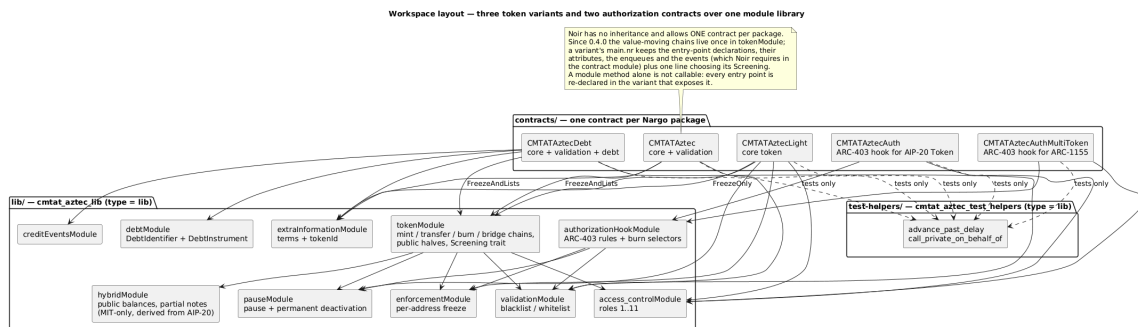


Diagram source: <doc/img/architecture.puml>.

What the token keeps private is the holder balances and the transfers between them. Everything an issuer needs to administer publicly — supply, roles, pause state, the compliance flags — stays public by design.

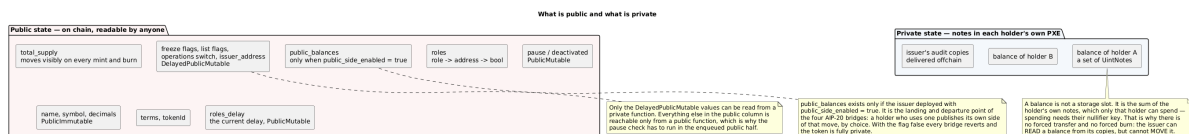


Diagram source: <doc/img/state-split.puml>.

Storage

- **Issuer_address:** `DelayedPublicMutable<AztecAddress, CHANGE_ROLES_DELAY_SECONDS>` - The address of the issuer, which receives a copy of every note and so can audit holder balances. It is a `DelayedPublicMutable` for two reasons: so that a *private* function can read it without a public call that would leak the caller, and so that it can be changed.
 - **It can be rotated by the admin**, with `set_issuer(new_issuer)` under `DEFAULT_ADMIN_ROLE`. The change is scheduled and becomes current after `CHANGE_ROLES_DELAY_SECONDS`; until then every mint, transfer and burn still addresses the previous issuer. The `IssuerChanged` event carries `effective_at`.
 - **Rotation transfers future visibility only.** Copies already delivered to the previous issuer cannot be recalled — a delivered note is delivered — so the previous issuer keeps what it has. The new issuer needs a PXE able to decrypt and store copies from the moment the change takes effect, or the audit trail has a hole for that period.
- **Public balances** (`public_balances: PublicBalances, a Map<AztecAddress, PublicMutable<u128>>`) and `public_side_enabled: PublicImmutable<bool>` — the AIP-20 public side, reachable only through the four bridges and only when the deployment flag is on. See [Private/public bridges](#).

- **Balances:** `Owned<BalanceSet>` - Token balance of every user inside their PXE, accessed as `private_balances.at(address)`. The balance of a user is the sum of the amounts of all their private `UintNote`. `BalanceSet` now comes from the `balance_set` aztec-nr library rather than being defined in this repository.

Mint private specifications

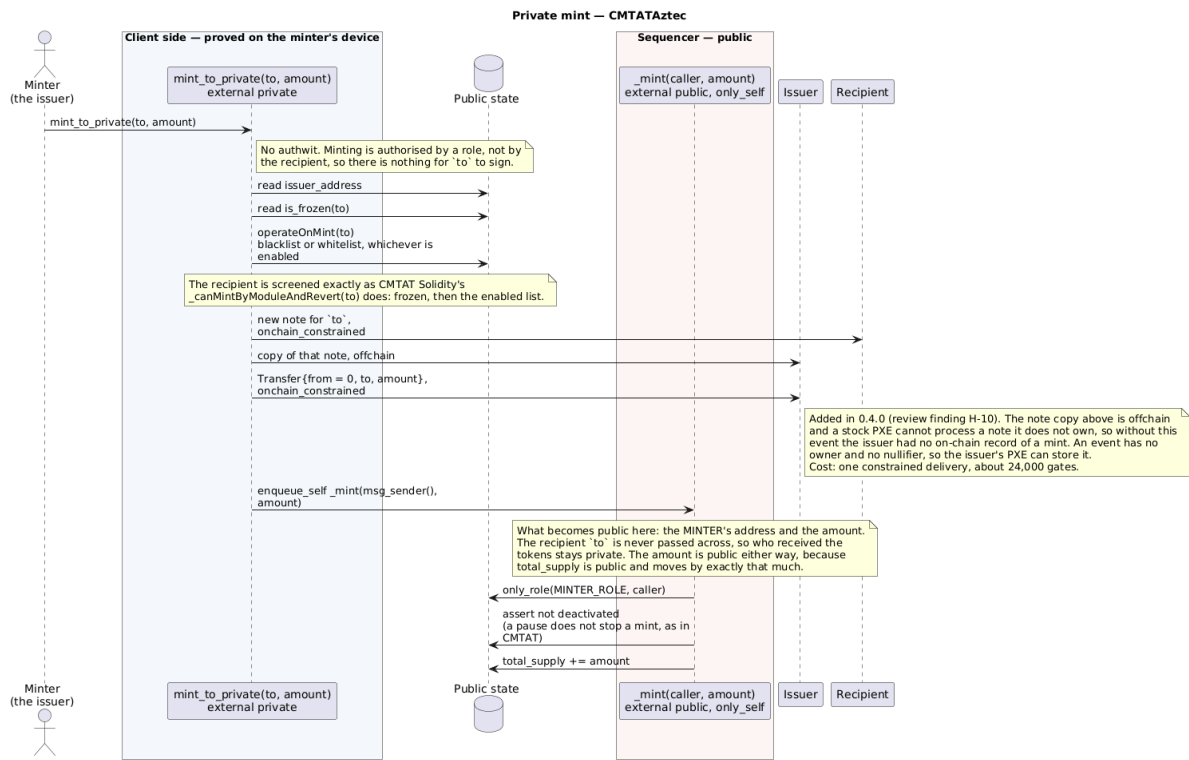


Diagram source: <doc/img/mint-flow.puml>.

Issuer:

- The new notes of the recipient are broadcasted to the issuer.

Failure cases:

- **Enforcement module:** If the `recipient` address is frozen, the mint will fail.
- **Validation module:** If a list mode is enabled, the `recipient` is screened against it — a blacklisted recipient, or one missing from the whitelist, fails the mint. Same as CMTAT Solidity's `_canMintByModuleAndRevert(to)`.
- **Authorisation module:** If the caller doesn't have the minter role, the mint will fail.
- **Pause module:** A pause does **not** stop a mint, as in CMTAT Solidity; deactivation does.

Limitations:

- `mint_batch` is capped at `MAX_ADDR_PER_CALL` recipients. See [Batching limits](#) for how that number was arrived at.

Transfer private specifications

The transfer is the flow worth reading closely: it shows the private half doing all the work on the user's own device, and the enqueued public half deliberately taking no arguments at all.

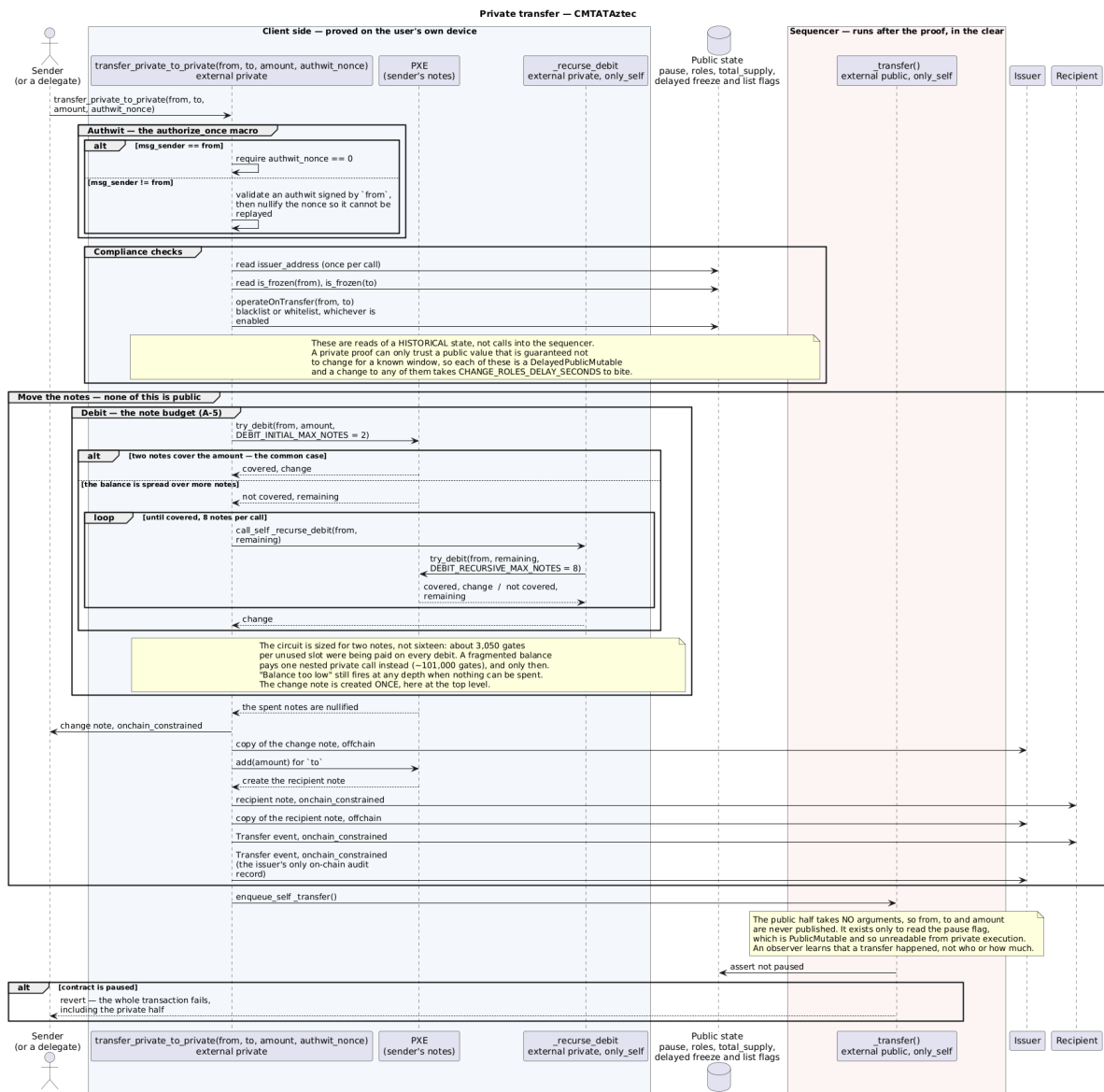


Diagram source: <doc/img/transfer-flow.puml>.

Issuer:

- The added notes from sender and recipient are broadcasted to the issuer.

Failure cases:

- Enforcement module:** If `from` or `to` addresses are frozen, the transfer will fail.
- Validation module:** If operations are enabled, the module checks if `from` or `to` should be restricted.
- Pause module:** If the contract is paused, the transfer will fail.

Limitations:

- `transfer_batch` is capped at `MAX_ADDR_PER_CALL` recipients, and transfer is the operation that sets that cap for all three. See [Batching limits](#).

The enqueued public half, `self.enqueue_self._transfer()`

A private function cannot read the pause flag: `is_paused` is a `PublicMutable`, and private execution runs on a historical snapshot on the user's own device. The check therefore has to happen in public, after the private proof, and that is what this one line arranges.

```
// in the private half, after the notes have moved
self.enqueue_self._transfer();

// the public counterpart, elsewhere in the same contract
#[external("public")]
#[only_self]
fn _transfer() {
    require_transfer(self.storage.pause_module);    // assert(!is_paused, "The
contract is paused")
}
```

How the call works.

- **enqueue_self is not a Noir feature.** The `#[aztec]` macro generates one method on it for each non-view public function of this contract, so `_transfer()` here is type-checked against the declaration above.
- **The generated method queues, it does not run.** It serialises the arguments, hashes them, stores the calldata in the execution cache and adds an entry to the transaction's public call stack.
- **The sequencer executes it after the private part.** A revert there reverts the whole transaction, private side effects included.
- **Nothing comes back.** An enqueued call has no return value, which is why the private half cannot branch on the pause state and simply relies on the transaction failing.

Why it takes no arguments. This is the single most important privacy property of a transfer, and it is enforced by a comment in each `main.nr` rather than by the compiler.

The enqueued call is public: its target contract, its function selector and every argument are visible on chain. `_transfer()` passes none, so an observer learns only that *some* transfer of this token happened — never `from`, `to` or `amount`. Adding a parameter here would publish it on every transfer.

The caller is visible too, which is what [H-3](#) weighed. The alternative was a delayed pause flag readable in private, rejected because a pause that takes effect hours later is not an emergency lever.

Where it is used. Three entry points enqueue it, all in each variant's `main.nr`:

Entry point	Why
<code>transfer_private_to_private</code>	the ordinary private transfer
<code>transfer_batch</code>	once for the whole batch, not once per recipient
<code>transfer_private_to_commitment</code>	the commitment is filled from private, so no public half of its own runs

The other bridges do not enqueue it, because their own public halves already apply the same rule: `_credit_public` and `_debit_public` call `require_transfer(pause)` before touching a public balance. Mint and burn enqueue `_mint` and `_burn` instead, which check *not deactivated* rather than *not paused* — a pause stops transfers only, as in CMTAT Solidity.

Burn private specifications

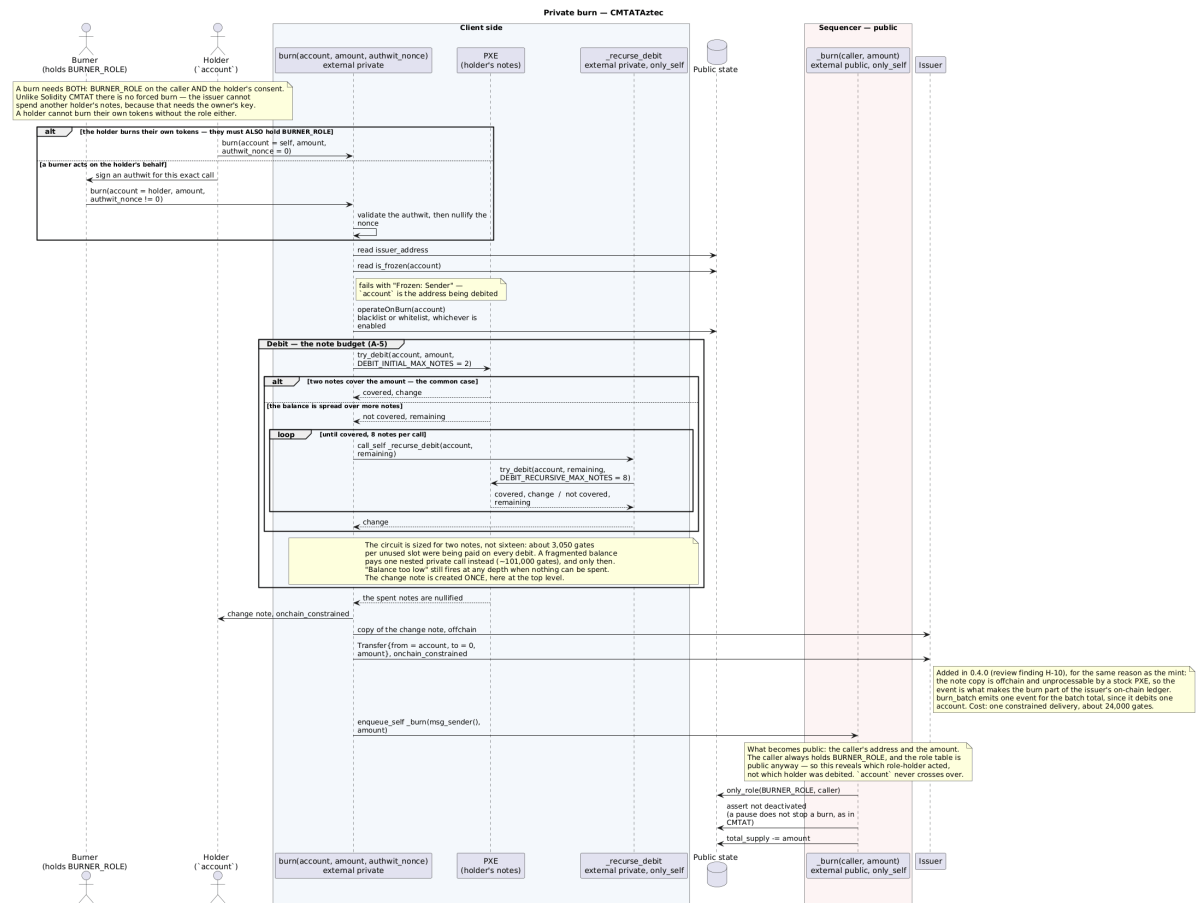


Diagram source: `doc/img/burn-flow.puml`.

Issuer:

- The new notes of the recipient (if any remaining) are broadcasted to the issuer.

Failure cases:

- Enforcement module:** If the `account` being debited is frozen, the burn will fail.
- Validation module:** If a list mode is enabled, the `account` is screened against it, as CMTAT Solidity's `_canBurnByModuleAndRevert (from)` does.
- Authorisation module:** If the caller doesn't have the burner role, the burn will fail.
- Pause module:** A pause does **not** stop a burn, as in CMTAT Solidity; deactivation does.
- Authwit:** If `from` doesn't issue an `AuthWit` the burn will fail

Note: The `AuthWit` issue is a key difference from Solidity smart contract logic, and users should be aware.

Limitations:

- `burn_batch` is capped at `MAX_ADDR_PER_CALL` entries, all debited from the same account. See [Batching limits](#).

Batching limits

`mint_batch` and `burn_batch` act on at most `MAX_ADDR_PER_CALL` addresses, currently **4**.
`transfer_batch` has its own, lower cap, `MAX_TRANSFER_ADDR_PER_CALL`, currently **2**.

Both numbers are measured, not derived from the protocol constants. Every value was tried by setting the global, adjusting the tests and running the full Noir suite.

Mint and burn — one note, its constrained delivery, and since 0.4.0 one constrained event to the issuer per recipient (mint) or per call (burn):

<code>MAX_ADDR_PER_CALL</code>	<code>mint_batch</code>	<code>burn_batch</code>	Result
1	30,776	81,736	all tests pass (measured before the issuer's mint / burn events)
2	60,169	160,189	all tests pass (same)
4	218,669	183,691	all tests pass
5	—	—	batched mint and burn end with a wrong total supply

Transfer — two notes and, since the `Transfer` event is delivered constrained to both the recipient and the issuer, **four constrained deliveries per recipient**:

<code>MAX_TRANSFER_ADDR_PER_CALL</code>	<code>transfer_batch</code>	Result
1	119,290 (= <code>transfer_private_to_private</code>)	all tests pass
2	228,274	all tests pass
3	—	aborts: <code>Assertion failed: push out of bounds</code>
4	—	aborts: <code>Assertion failed: push out of bounds</code>

Gate counts are from `aztec profile gates` on `CMTATAztec` and are the current code. The mint and burn figures include the list screening on the target (about 6,200 gates per address); the transfer figures include both constrained event deliveries (about 20,200 gates each).

Three things are worth drawing out of those tables.

- **Transfer's cap is set by its deliveries, not its notes.** With the `Transfer` event delivered constrained to two parties, a recipient costs four constrained deliveries where a mint costs one. Before the event was constrained, a 4-recipient batch passed; with it, 3 already fails. A model consistent with every measurement is that each constrained delivery consumes two of the sixteen key-validation requests a call may make — $4 \times 2 \times 2 = 16$ fitted, $3 \times 4 \times 2 = 24$ does not — but the cap is the measurement, not the model.
- **The number of nested private calls is irrelevant.** The chains (`tokenModule::mint_private` / `transfer_private` / `burn_private`) are ordinary library functions, inlined by the compiler exactly as the former `#[internal("private")]` helpers were: a batch makes no nested private calls at all, whatever the cap is. Earlier revisions of this document cited the 8-private-call limit as a constraint on batching; it never was one.
- **Batching moves work, it does not remove it.** A 2-recipient transfer is a 228,274-gate circuit against 119,290 for a single transfer — and that proof is produced on the *user's own device*. What batching saves is the fixed per-transaction protocol overhead, which two separate

transfers would pay twice. Batch because you want one transaction, not because you want a cheaper circuit.

Raising either cap means repeating the measurement, not re-reading the protocol constants. It is also an ABI change: the array lengths in `mint_batch`, `transfer_batch` and `burn_batch` are part of the generated interface. The transfer cap in particular was **lowered** from 4 to 2 by the decision to deliver the `Transfer` event constrained — see [Events](#) for why that trade was taken.

How many notes a transfer spends, and what it costs. A debit does not size its circuit for every note a holder could have. It tries `DEBIT_INITIAL_MAX_NOTES` (2) notes first; if the holder's balance is spread over more, the contract calls its own `#[only_self]` entry point `_recurse_debit`, which spends up to `DEBIT_RECURSIVE_MAX_NOTES` (8) more and calls itself again if needed, each call with its own side-effect budget. The scheme and both values are AIP-20's (`_subtract_balance` / `recurse_subtract_balance_internal` in the `aztec-standards` token); the code is in `tokenModule.nr` (`try_debit`, `debit_private`, `debit_recursive`). What it changes, measured (review A-5):

- **A holder with one or two notes pays a smaller circuit:** `transfer_private_to_private` 161,493 → **119,290** gates (~26%), `burn` 111,638 → 69,434, `transfer_private_to_public` 95,941 → 53,737, `transfer_private_to_commitment` 93,063 → 50,857, `transfer_batch` 312,909 → 228,274, `burn_batch` 352,719 → 183,691 (base and Debt; Light about 10,000 lower on each). Before, every debit was compiled for 16 notes at about 3,050 gates per slot, used or not.
- **A fragmented balance pays a recursive call instead of failing.** Three to ten notes cost one `_recurse_debit`: 30,138 gates for its own circuit, plus a kernel iteration for the nested call. Every further eight notes cost one more. The ceiling of twelve notes per transfer, and the failure past sixteen, are gone: fifty notes in one transfer were measured to pass; the remaining bound is the protocol's nested-call limit.
- **A wallet can still consolidate** with transfers to self when it wants a holder's next transfer to be the cheap case, but it no longer has to.

Events

Every operation that changes contract state emits an event. Most are **public** — plain logs anyone can read, mirroring the OpenZeppelin and CMTAT events of the Solidity implementation. One, `Transfer`, is **private**: encrypted and delivered to named recipients, because its content is exactly what this token keeps confidential.

Events must be declared inside the contract module, not in the shared library, which is why each variant's `main.nr` re-declares them.

Public events

Event	Fields	Emitted by	Reference
<code>NewRole</code>	<code>role</code> , <code>account</code>	<code>grant_role</code> , and the constructor for its two founding grants	OpenZeppelin <code>RoleGranted</code>
<code>RoleRevoked</code>	<code>role</code> , <code>account</code> , <code>sender</code>	<code>revoke_role</code> , <code>renounce_role</code>	OpenZeppelin <code>RoleRevoked</code>
<code>Paused</code> / <code>Unpaused</code>	<code>account</code>	<code>pause_contract</code> / <code>unpause_contract</code>	OpenZeppelin
<code>Deactivated</code>	<code>account</code>	<code>deactivate_contract</code>	CMTAT
<code>AddressFrozen</code>	<code>account</code> , <code>is_frozen</code> , <code>enforcer</code> , <code>effective_at</code>	<code>freeze</code> , <code>unfreeze</code>	CMTAT <code>AddressFrozen</code>

Event	Fields	Emitted by	Reference
<code>AddressListed</code>	<code>account</code> , <code>is_blacklisted</code> , <code>is_whitelisted</code> , <code>operator</code> , <code>effective_at</code>	<code>add_to_list</code> , <code>remove_from_list</code>	— (CMTAT's is allowlist-specific)
<code>OperationsSet</code>	<code>operate_blacklist</code> , <code>operate_whitelist</code> , <code>operator</code> , <code>effective_at</code>	<code>set_operations</code>	≈ CMTAT <code>AllowlistEnableStatus</code>
<code>IssuerChanged</code>	<code>issuer</code> , <code>operator</code> , <code>effective_at</code>	<code>set_issuer</code>	—
<code>RolesDelayChanged</code>	<code>new_delay</code> , <code>operator</code> , <code>effective_at</code>	<code>set_roles_delay</code>	—
<code>Terms</code>	<code>name</code> , <code>uri</code> , <code>documentHashHigh</code> , <code>documentHashLow</code> , <code>lastModified</code>	<code>set_terms</code>	CMTAT <code>Terms</code>
<code>TokenId</code>	<code>tokenId</code>	<code>set_token_id</code>	CMTAT <code>TokenId</code>
<code>DebtLogEvent</code> , <code>DebtInstrumentLogEvent</code> , <code>CreditEventsLogEvent</code>	<code>account</code>	<code>set_debt</code> , <code>set_debt_instrument</code> , <code>set_credit_events</code> (CMTATAztecDebt only)	CMTAT, which emits them payload-free; these carry the caller
<code>Transfer</code> (public)	<code>from</code> , <code>to</code> , <code>amount</code> , <code>with</code> <code>PRIVATE_ADDRESS_MAGIC_VALUE</code> for the private side	<code>transfer_private_to_public</code> (<code>from</code> = marker), <code>transfer_public_to_private</code> (<code>to</code> = marker) — the bridges only	AIP-20 <code>Transfer</code> and its <code>PRIVATE_ADDRESS</code> sentinel

`effective_at` appears on every event for a `DelayedPublicMutable` value. It is the timestamp from which the scheduled value is current — exactly what the state variable records, computed as block timestamp plus the module's delay — so an indexer does not need to know the contract's delay to know when a freeze, a listing, an operations change or an issuer rotation takes effect.

The private event

Event	Fields	Emitted by	Delivered to	Mode
<code>Transfer</code>	<code>from</code> , <code>to</code> , <code>amount</code>	<code>transfer_private_to_private</code> , and <code>transfer_batch</code> once per recipient	the recipient and the issuer	<code>onchain_constrained</code> , both
<code>Transfer</code> with <code>from</code> = 0	<code>from</code> = <code>AztecAddress::zero()</code> , <code>to</code> , <code>amount</code>	<code>mint_to_private</code> , and <code>mint_batch</code> once per recipient	the issuer	<code>onchain_constrained</code>
<code>Transfer</code> with <code>to</code> = 0	<code>from</code> , <code>to</code> = <code>AztecAddress::zero()</code> , <code>amount</code>	<code>burn</code> (per call) and <code>burn_batch</code> (one event for the batch total, since it debits one account)	the issuer	<code>onchain_constrained</code>

This one is worth explaining, because both choices — who receives it, and how — were made deliberately and cost something.

What it uniquely provides. A transfer already delivers two constrained notes: the sender's change note and the recipient's new note, each copied offchain to the issuer. A note is a value and an owner; it has no sender field. So everything the event says is already known to someone from the notes — the amount, the recipient, the sender to the issuer by correlating its two copies — **except one thing: the sender's identity, to the recipient.** That is the event's job here: a receipt saying who paid you. It is not an indexer feed, as `Transfer` is in Solidity; it is encrypted.

Why constrained. An `onchain_unconstrained` delivery is "on-chain delivery without constrained encryption": the circuit computes `from` correctly, but nothing proves that what the sender's PXE posts encrypts that value. The recipient would decrypt whatever the sender chose — a receipt the sender can forge is not a convenience but a settlement-confirmation attack surface. Constrained delivery makes the receipt provable. It costs about **20,200 gates per delivery**, measured.

What "constrained" constrains, precisely: the AES-128 encryption itself. This is worth stating because it is easy to assume the cost buys only the tag. Under `onchain_constrained()` the framework guarantees that "both the encryption and the discovery tag are constrained and stored onchain", and charges "proving time overhead for encryption and tagging" (aztec-nr's `note_delivery` documentation). The implementation bears that out: `aztec-nr`'s `messages/encryption/aes128.nr` encrypts with `std::aes128::aes128_encrypt`, a Noir standard-library function, so it compiles to constraints in the circuit — while *decryption* runs through an oracle, `try_aes128_decrypt`, and is therefore unconstrained. The asymmetry is the right way round: the sender must prove it encrypted correctly, and the recipient's PXE only has to try keys during discovery, where nothing needs proving.

Two clarifications that follow from it. The constraints live in **this contract's own circuit**, not in the protocol's kernel circuits — which is why the 20,200 gates show up per function in `aztec_profile_gates`, and why AES-128 is unavailable in public (AVM) functions while working in private ones. And the guarantee applies only to the constrained mode:

`onchain_unconstrained()` and `offchain()` both encrypt *without* constraints, so the issuer's offchain audit copies carry no such proof — the reason the `Transfer` events, not the note copies, are the audit trail.

Why the issuer. The issuer's note copies are offchain, and a stock PXE cannot store a note it does not own by either delivery mode (see *Limitations*). Without the events, three things followed:

- the issuer's whole audit trail had no data availability;
- a dropped message was undetectable;
- and even a delivered copy was unprocessable.

An event has no owner and no nullifier, so the issuer's PXE validates its commitment against the tree and stores it without anyone else's keys.

The `Transfer` stream is therefore the issuer's on-chain, unforgeable **ledger**. Since 0.4.0 it covers every movement of the fully private token — mints (`from = 0`) and burns (`to = 0`) as well as transfers and their batches, following the ERC-20 and AIP-20 convention — so replaying it reconstructs every holder's balance. No set of note copies could do that: a copy says a note was created, never that it was spent. The copies remain as corroboration, the preimage of a note the ledger says exists.

Two paths are outside that stream, and both belong to the public side. They exist only in a token deployed with `public_side_enabled = true`; for a fully private deployment the paragraph above is the whole story.

- **The bridges** publish a `Transfer` naming their public party and the sentinel `PRIVATE_ADDRESS_MAGIC_VALUE` for the private one, as AIP-20 does. The movement is recorded, the private counterparty is not, so it is the public balance rather than the private one that a replay reconstructs here.
- **A commitment completion emits no `Transfer` at all.** It cannot: at completion the contract holds the commitment and never the recipient's address, and an event naming a recipient

the contract could not verify would be forgeable by the payer — the opposite of what constrained delivery is for. The issuer reconstructs these from the `CommitmentInitialized { to, completer, commitment }` event it receives when the commitment is opened, deriving the completion log's tag from the commitment and reading the amount there; see [Private/public bridges](#).

Both facts are pinned mechanically rather than left to this prose. A warm `transfer_private_to_private` leaves four private logs — one per note, plus the event's two constrained deliveries — and a `transfer_private_to_commitment` of the same value leaves the same two notes and two logs, no event among them. `tests/cmtat-aztec/src/test_invariants.nr` asserts both counts, so either path gaining or losing an event fails the suite.

What the mint and burn records cost. One constrained delivery each: `mint_to_private` 36,976 → 61,062 gates, `burn` 87,935 → 111,638, `mint_batch` 132,584 → 218,669 (one event per recipient), `burn_batch` 331,362 → 352,719 (one event for the total) on the base and Debt variants; Light: 30,776 → 54,862, 81,736 → 105,439, 107,871 → 193,956, 306,820 → 328,177. The batch caps are unchanged: `mint_batch` at 4 recipients carries eight constrained deliveries, the same count `transfer_batch` at 2 already carries.

What it cost. `transfer_private_to_private` went from 120,824 to **161,493 gates** (+34%; 119,290 since the note budget of A-5 took 43,046 back), and because each constrained delivery counts against a per-call budget, the transfer batch cap fell from **4 to 2** recipients — see [Batching limits](#). That trade was taken knowingly: a security token's audit trail is the point of the instrument, and batched transfers are its rare path.

What is not public. The event is encrypted to its two recipients. An outside observer sees that private logs exist, padded like every other private log, and learns nothing about the parties or the amount — see [What each operation publishes](#).

Security and confidentiality properties

What each operation publishes

Every private operation enqueues one public call, and **every argument of a public call is public**. What crosses that boundary is therefore the whole of what an outside observer learns; the three operations were designed so that as little as possible does:

Operation	Public callee	Published in the clear	Kept private
<code>mint_to_private(to, amount)</code>	<code>_mint(caller, amount)</code>	the minter's address, the amount	the recipient <code>to</code>
<code>transfer_private_to_private(from, to, amount, ...)</code>	<code>_transfer()</code> — no arguments	that a transfer of this token occurred	sender, recipient, amount
<code>burn(account, amount, ...)</code>	<code>_burn(caller, amount)</code>	the burner's address, the amount	the debited <code>account</code>

Operation	Public callee	Published in the clear	Kept private
<code>transfer_private_to_public(from, to, amount, ...)</code>	<code>_credit_public(to, amount)</code>	the public recipient and the amount — the sender chose to pay a public balance	the sender <code>from</code>
<code>transfer_public_to_private(from, to, amount, ...)</code>	<code>_debit_public(from, amount)</code>	the public sender and the amount — its balance was public already	the recipient <code>to</code>
<code>transfer_private_to_commitment(from, commitment, amount, ...)</code>	<code>_transfer()</code> — no arguments	that a transfer occurred; the amount , unencrypted in the completion log, tagged by the commitment	sender, recipient
<code>initialize_transfer_commitment(to, completer)</code>	none	nothing (the validity commitment is a nullifier)	<code>to</code> , <code>completer</code>

Three things follow, and they are worth stating precisely because the obvious reading of the table overstates the leak.

- **The amounts were public anyway.** `total_supply` is a `PublicMutable<u128>` that moves by exactly the minted or burned amount in the same transaction, so passing `amount` to the public half reveals nothing the supply change does not. This is a consequence of the design decision to keep the supply public, recorded under *Assumptions*.
- **The published address always holds a role.** `_mint` and `_burn` publish `msg_sender()` because they must check `MINTER_ROLE` / `BURNER_ROLE` on it, and the role table is public state anyone can enumerate. So the marginal disclosure is *which* role-holder acted and *when* — not a new identity. Where the issuer is the sole minter and burner, which is the expected deployment, this amounts to a public issuance-and-redemption ledger keyed to the issuer, which is arguably what a security token wants.
- **The holder is not published.** `to` in a mint and `account` in a burn stay in the private half. A burn executed by the issuer under a holder's authwit publishes the issuer, not the holder.

The caveat that matters: do not grant `BURNER_ROLE` to holders. The privacy of a burn rests entirely on the burner and the holder being different parties. The moment a holder holds `BURNER_ROLE` and redeems its own tokens, the published burner *is* the holder, and that burn — address and amount — is fully public. The same applies to `MINTER_ROLE` and self-minting. These roles are issuer roles by design; delegating them to holders turns a private operation into a public one without any code changing.

The public callee's **selector** also distinguishes the three operations from each other — an observer can tell a mint from a burn from a transfer. For transfer that reveals only "a transfer happened"; for mint and burn it composes with the two rows above. There is no cheap fix: hiding the selector would mean one shared public function taking the operation kind as an argument,

which publishes the same fact one level down, and would newly publish the caller on transfers.

The one design that would remove the transfer selector is to make the pause flag a

`DelayedPublicMutable`, as the freeze and list flags already are, so that

`transfer_private_to_private` could check it in private and enqueue nothing. That was considered and **rejected**, for three reasons that compound:

- A scheduled pause takes effect only after the delay, which is the opposite of what a pause is for.
- Every private read of a delayed value sets the transaction's `expiration_timestamp`, so a delay short enough to be useful as a pause would give every transfer a validity window of minutes and an expiration offset unique to this contract.
- The library's own recommendation is a delay of hours, which it calls unsuitable for an emergency shutdown. For a security token the pause is the emergency lever and must take effect in the next block, so `is_paused` stays a `PublicMutable<bool>` checked in `_transfer`, and the public call that reveals "a transfer of this token occurred" is the price. Analysis finding [H-3](#) records the four options and their costs.

What this public call does to the delay's privacy argument.

- **The leak the delay normally carries.** A private read of a delayed value sets the transaction's public `expiration_timestamp` to `anchor + delay`. That tells an observer "this transaction read a value with *that* delay", narrowing it to the contracts using that delay — which is why the library recommends aligning delays with common values, and why [Delay of the delayed values](#) discusses the neighbourhood.
- **Here it is dominated.** Every value-moving operation already enqueues a public call whose target is this contract, so an observer knows "CMTAT token, transfer" before looking at any expiry. Knowing the delay adds nothing about *which* contract, so the delay buys no anonymity set at any value, as long as the pause is checked in public.
- **What the delay still governs.** The validity window — prove and include within it — and the freeze / listing latency. Those are what its value is chosen on.
- **The one leak it carries at any value.** Timing: `expiration_timestamp - delay` is the anchor block the sender proved against.

The two decisions are one. An immediate pause spends the privacy the delay could have protected; a delayed pause would give it back at the price of a slow emergency lever. The same holds for the authorization contracts, whose hook enqueues one public call per transfer.

• Private mint call to public function:

- **Reveals minter address:** Since it is a parameter in the public function call. It is the issuer, whose address is already known, but still, private to public function calls pose a problem as they also reveal that the contract was called.
- **Randomizing `msg.sender`:** An out-of-protocol option is to deploy a diversified account contract and route transactions through this contract. Application developers might also do something similar to randomize the `msg.sender` of their app contract's address.
- **Leakage of minted amount:** The amount being minted is leaked as it is passed to the public function from the private one.

In the case of our token, when an issuer mints tokens, it is publicly known how much tokens he mints. This means that if the issuer mints “on-demand” (every time a user wants to mint some tokens, the issuer mints) then there is a leak of information. This can be mitigated by the issuer minting a fixed amount of tokens at a certain point in time (= circulating supply), and then privately distributing to the users, thus revealing way less information.

- **Traceability:** The public transaction will be traceable back to the private proof.
 - **Disclosure of private function call:** It will leak that a private function (`private_mint`) has been called.
 - **Recipient address privacy:** It will **not** leak the address to which this amount is being sent.
- **Note encryption constraints:**
 - Note encryption should be **constrained**. We could make note encryption and tagging unconstrained, as this is allowed, but we don't want to.
 - **Incentive alignment:** Unconstrained note encryption is done when the sender has an incentive to send correct information to the receiver, as no one proves and verifies it. However, in our case, the sender is in no way incentivized to do the right thing.
 - **Optimization:** For optimization purposes, unconstrained might be acceptable in some places.

Modules

Aztec Noir uses Rust-like modularity, which means that there is no Solidity-like abstract contract and inheritance. Instead, we use separated modules in the form of interfaces and implementations. Every function that can or should be called by a user needs to be exposed in the main contract. Consequently, not everything can be displaced from the main contract — `mint_to_private`, `burn` and `transfer_private_to_private` all live there — and most functions are exposed there.

Token module - Shared Context

`lib/src/modules/tokenModule.nr` holds the value-moving chains once for the three variants:

- **The three operations** — `mint_private`, `transfer_private`, `burn_private`: screening, then the note movement with the issuer's copy of every note.
- **The bridge chains** — `bridge_private_to_public`, `bridge_public_to_private`, `open_commitment`, `pay_commitment`.
- **The batch preconditions, and the public halves' bodies** — `mint_public`, `burn_public`, `require_transfer`, `credit_public`, `debit_public`.

The functions take the state variables as arguments and never own them, so each contract keeps its own `#[storage]` and slot layout.

What a variant screens is a `Screening` value it builds from its own storage: `FreezeAndLists` for `CMTATAztec` and `CMTATAztecDebt`, `FreezeOnly` for `CMTATAztecLight`. That one library method, `screening(storage)`, is the only place the three contracts differ on these paths.

Each `main.nr` keeps what Noir requires in the contract module: the `#[external]` declarations, their attributes, the `enqueue_self` calls and the event emissions.

The refactor's rationale and measured effect are in [doc/technical/token-module.md](#). The gate profile is identical before and after, because both `#[internal]` helpers and library functions are inlined.

Authorisation module (access control) - Public Context

- This module is used by other modules and by the `mint_to_private` and `burn` functions.
- Modules only need to call the `only_role` function, which publicly verifies if an address has sufficient roles for the action; otherwise, it reverts.
- The default role is the `DEFAULT_ADMIN_ROLE`, which can grant other roles.
- **Implementation note:** This module's implementation is quite cumbersome, as in the main contract, an instance of this module is passed to each function call. This is because the object is unique, and we cannot pass it as a context (at least until a working implementation is found).

Validation module - Shared Context

- This module is called only when performing transfers.
- The `operateOnTransfer` function, used in a private context, is called by the transfer function.
- Each user flag update is delayed by the current setting, `roles_delay()` — `CHANGE_ROLES_DELAY_SECONDS` (one hour) at deployment, adjustable afterwards; see [Delay of the delayed values](#).
- If no operations are enabled, no checks are done, but the function is still called.
- Operations can be enabled or disabled, and there is also a delay.
- Currently, no operations can be added; there is only blacklist/whitelist.

Delay issue:

The diagram below is the whole argument in one picture: why the flags must be delayed, and what that delay costs.

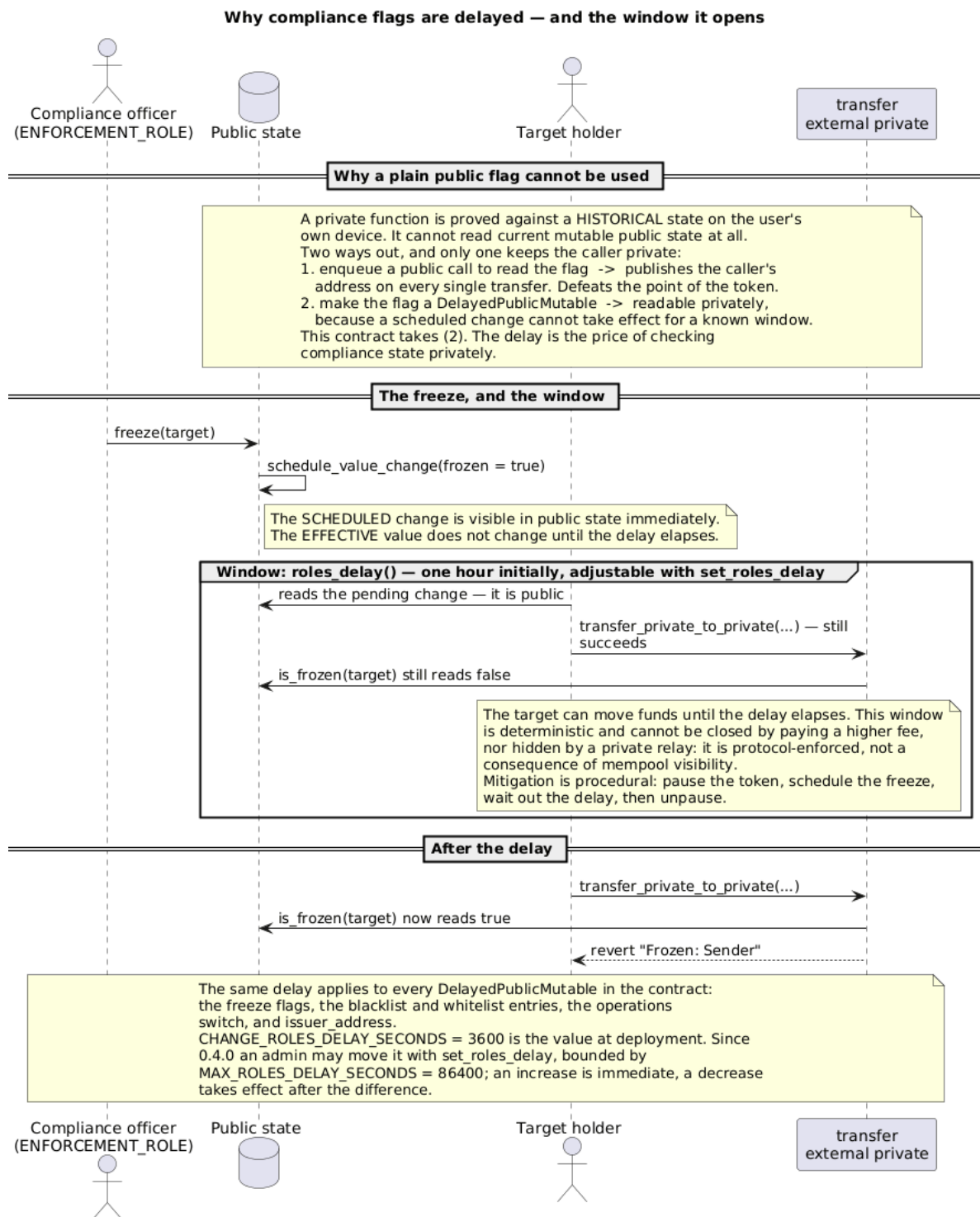


Diagram source: <doc/img/delayed-flag.puml>.

- The delay is caused by the flags being stored in a `DelayedPublicMutable`. To be precise about which state this is: the **validation flags** (`users`, `operationsFlag`), the **freeze flags** and `issuer_address`. The **roles** are not delayed — `RoleData.has_role` is a `Map<AztecAddress, PublicMutable<bool>>` and a grant or revoke takes effect in the same block. The constant is named `CHANGE_ROLES_DELAY_SECONDS` for historical reasons, which is misleading on this point.
- This is needed to preserve privacy when doing a private transfer between two users while maintaining the strict rule that no tokens should be transferred from/to a blacklisted address.
- **Problem:** A user who knows they are going to be blacklisted before the delay elapses might send their funds to an address that is not blacklisted. This problem has no solution for now,

and the target does learn of a pending freeze — see the [FAQ](#) for the three public sources that tell it.

- **Consideration:** We need to think about whether the shared state will be changed often. If not, then `DelayedPublicMutable` is an acceptable solution; otherwise, it might be problematic.

Potential solutions:

- **Theoretical solution 1:** Using a `DelayedPublicMutable` is essential because otherwise, you would use a `PublicMutable`, which means that the user calling the transfer function needs to call a public function to read the `PublicMutable` variable, leaking the sender's address. One possible solution might be to hide the caller's address using [Diversified and Stealth Addresses](#). If reading `PublicMutable` did not leak the user address, then `DelayedPublicMutable` would be unnecessary.
- **Theoretical solution 2:** Have a counter that is set when the `DelayedPublicMutable` is changed. For the `COUNTER` amount of time, the token contract is paused to prevent any blacklisted address from retrieving funds. This solution is poor in terms of user experience and developer experience, as the issuer needs to manually unpause the contract.
- **Practical solution 3:** If we whitelist instead of blacklist, a new whitelisted address will not be able to transfer funds directly, which is not a significant issue.

What is implemented today. Of the three above, one is in the contract and one is available by hand:

- **Solution 3 is implemented and is the issuer's choice at runtime.** The validation module carries both modes — `BLACKLIST_FLAG` and `WHITELIST_FLAG`, held together in a `SetFlag { operate_blacklist, operate_whitelist }` — and `set_operations (VALIDATION_ROLE)` turns either or both on. In whitelist mode the delay works in the safe direction: a newly listed address simply cannot transfer until the delay elapses, whereas in blacklist mode a newly listed address can still move funds during it. An issuer that cannot accept the escape window should run in whitelist mode; that is the mitigation, and it costs the delay on every new holder instead.
- **Solution 2 is not implemented as a counter, but the pause it needs exists and is immediate.** `is_paused` is a `PublicMutable<bool>` checked in the enqueued `_transfer`, deliberately not delayed (see [Delay of the delayed values](#)), so an issuer that wants the "freeze the whole token while a listing takes effect" behaviour can call `pause_contract`, then `add_to_list`, then `unpause_contract` after the delay. Nothing automates the sequence, and the manual unpause is exactly the operational cost the solution describes.
- **Solution 1 is not available.** It depends on reading a `PublicMutable` from private without revealing the caller, which the protocol does not offer at 5.2.0; diversified and stealth addresses remain a specification, not an implementation.

What changed since these were written. The delay is no longer a compile-time constant:

`set_roles_delay(new_delay) (DEFAULT_ADMIN_ROLE, 1 ≤ new_delay ≤ 86400)` moves it at runtime, the per-address entries adopt the setting when they are next written, and each event carries the `effective_at` the library scheduled, not `now + delay`. That does not close the escape window — it lets the issuer trade its width against the transaction-validity window and the privacy set, and the **Consideration** bullet above is the decision it asks for.

Pause module - Public Context

- The pause module is a `PublicMutable`.
- The functions to set and unset the pausable flag are protected under Access Control.
- The pause check is done in public state, in the enqueued half of `transfer_private_to_private`. As in CMTAT Solidity, `mint_to_private` and `burn` are not stopped by a pause; their enqueued halves check deactivation instead, so a deactivated token (which is paused forever) can do none of the three.

Enforcement module - Shared Context

- This module is called in `mint_to_private`, `transfer_private_to_private`, and `burn` to check if an address has been frozen.
- Unlike the validation module, this module is mandatory.
- Changing an address to frozen has a delay, as the value is a `DelayedPublicMutable`; see [Delay of the delayed values](#) for the value and how it is changed.

"Freeze Address" Note: The enforcement has a delay, similar to the validation module, and the target can see the freeze coming during it (see [FAQ](#)). One approach is to pause the contract before freezing some accounts for the delay time, then unpause it. This requires manual pause/unpause.

Delay of the delayed values

Four things are `DelayedPublicMutable`, so that a private function can read them: the freeze flag of each address, the list flags of each address, the list mode (`operationsFlag`) and the issuer address. A write to any of them is *scheduled* and becomes current only after a delay.

That one number has three effects:

- **Enforcement latency.** A freeze or a listing bites only after the delay.
- **A validity window.** Every transaction that read one of these values in private must be included within the delay of its anchor block, because the read sets the transaction's `expiration_timestamp`.
- **A public fingerprint.** That expiration is published, so it says which delay the transaction's contract uses.

Every value-moving entry point reads the issuer address and at least one flag, so the token's delay is the minimum over its variables — which is why they all carry the same one. Review finding [H-6](#) explains the trade in full.

The value is one hour initially (`CHANGE_ROLES_DELAY_SECONDS = 3600`), a middle position: the framework's own compliance token uses 24 hours, its authorisation example 360 seconds, and the library recommends "at least a couple hours". One hour leaves a proving-and-inclusion budget a phone can meet, keeps the freeze window to an hour, and can be moved without redeploying:

- `set_roles_delay(new_delay)` (`DEFAULT_ADMIN_ROLE`, $1 \leq \text{new_delay} \leq 86400$, the protocol's transaction lifetime) changes the setting and emits `RolesDelayChanged { new_delay, operator, effective_at }`. The library makes an **increase effective at once** and a **decrease effective only after the difference** between the old and new delay has elapsed, so nothing already scheduled can land earlier than it promised.
- The issuer address and the list mode are single variables and adopt the new setting in that call. The **per-address entries** (freeze and list flags) each carry their own delay, which the

contract cannot change from outside: `freeze`, `unfreeze`, `add_to_list` and `remove_from_list` apply the current setting to the entry they write before scheduling the value, so entries converge to the setting as they are touched. Until an entry has been written again it keeps its previous delay, and the `effective_at` in every event is the timestamp the library actually scheduled, not `now + setting`.

- `roles_delay()` returns the setting new writes adopt.

The pause is deliberately **not** delayed: it is a `PublicMutable` checked in the enqueued public half, so it takes effect at once and has no delay to adjust (see *Pause module*).

Both test suites clear the delay by moving the clock rather than waiting. The Noir suite does it through the TXE; the e2e suite warps a local network's L1 with

`RollupCheatCodes.advanceToSlot (src/utlis/time_travel.ts)` and falls back to waiting only against a real network.

At 24 hours the token's transactions would be indistinguishable from those of a contract that reads no delayed value at all, which is the largest privacy set available. The hour is the compliance side's price for a shorter freeze window, and the setting exists so that the issuer can move it in either direction once it knows how its holders prove and what its neighbours use.

Issuer's view of transactions and notes

- **Objective:** Enable the issuer to see all transactions.
- **Current implementation:** Note emission is duplicated: one message for the owner of that note, and a second copy of the same message for the issuer (`deliver_to(issuer, ...)`).
- **Delivery mode of the issuer's copy:** the owner's copy is delivered onchain and constrained; the issuer's copy is delivered **offchain**. Aztec's own documentation presents an onchain constrained copy to an auditor as the supported pattern, but PXE cannot process an onchain note message addressed to someone who is not the note's owner: note discovery computes the note's nullifier, which needs the owner's nullifier key. Delivering the issuer's copy offchain avoids the onchain cost of a copy the PXE cannot use, but it does **not** make the copy processable: the offchain path runs through the same discovery code, which skips any note whose nullifier it cannot compute. The 0.4.0 review traces this to `attempt_note_nonce_discovery` under H-10, and the framework tracks storing such notes as future work. The copies are decryptable by the issuer with custom tooling and have no onchain data availability - the issuer must capture them as they are produced, and a sender who drops them is not detectable onchain. The issuer's processable, on-chain record is the constrained `Transfer` event stream, which since 0.4.0 covers mints and burns as well as transfers (see *Events*).
- **Other potential implementations:**
 - **App-siloed key:** Use an app-siloed key that the issuer can use for decrypting any note in the note hash tree of this app.

Private/public bridges

Since 0.4.0 the token carries the four AIP-20 entry points that move value between a holder's private notes and a **public balance**, with the same names, parameter types and selectors as the standard. They exist so that a holder can interact with the public side of Aztec — a contract that keeps public balances, a vault, a settlement counterparty — without leaving the token, and they

are the holder's choice per transfer: what a bridge publishes is the side of the transfer that the mover chose to make public, never the counterparty's.

They are **off unless the issuer enables them at deployment**: the constructor's last argument, `public_side_enabled`, is a `PublicImmutable<bool>`; with `false` every bridge reverts with `Error: public side disabled at deploy` before any note is touched, and the token is exactly the fully private token described above. The flag cannot be changed afterwards.

Entry point	What it does	Published	Kept private
<code>transfer_private_to_public(from, to, amount, authwit_nonce)</code>	spends <code>from</code> 's notes, credits <code>to</code> 's public balance	<code>to, amount, Transfer(PRIVATE_ADDRESS, to, amount)</code>	<code>from</code>
<code>transfer_public_to_private(from, to, amount, authwit_nonce)</code>	debits <code>from</code> 's public balance, creates a note for <code>to</code> (with the issuer's copy)	<code>from, amount, Transfer(from, PRIVATE_ADDRESS, amount)</code>	<code>to</code>
<code>initialize_transfer_commitment(to, completer) -> commitment</code>	opens a partial note owned by <code>to</code> that only <code>completer</code> may fill	nothing	<code>to, completer</code>
<code>transfer_private_to_commitment(from, commitment, amount, authwit_nonce)</code>	spends <code>from</code> 's notes and fills the commitment; the caller must be its completer	the amount, unencrypted in the completion log, tagged by the commitment; that a transfer occurred	<code>from, to</code>
<code>transfer_private_to_public_with_commitment(from, to, amount, authwit_nonce) -> commitment</code>	<code>transfer_private_to_public</code> plus a commitment for <code>to</code> that the sender may fill later	as <code>transfer_private_to_public</code>	<code>from, the commitment's owner</code>
<code>balance_of_public(owner)</code>	reads a public balance	—	—

Each bridge enforces the same compliance chain as `transfer_private_to_private` — freeze and list screening, the issuer's copy of every note, and the pause check — divided across its two halves:

- **The private half** screens both parties' freeze flags and the enabled list, then delivers the issuer's copy of every note it creates.
- **The public half** asserts the contract is not paused, and nothing else.
- **A commitment is screened when it is opened, not when it is paid.** At completion the contract holds only the commitment and never the recipient's address, so there is nothing left to screen against. The issuer learns which holder a commitment belongs to from a `CommitmentInitialized { to, completer, commitment }` event delivered constrained to it, because the library delivers the partial note to `to` alone. That delivery is pinned by a test, so it cannot be dropped while the suite stays green.

Two things are deliberately absent:

- **No expiry on commitments.** A recipient frozen after opening one can still be paid into it, until the freeze is checked at the next bridge that recipient uses.
- **No public-to-public transfers, public mints or public burns.** A public balance is a landing and departure point, not a second ledger.

A commitment can be paid exactly once — a difference from AIP-20. In the standard, `PartialUintNote::complete` (aztec-nr) does not consume the commitment, and its documentation says why that matters: "the recipient only discovers the first completion, so anything carried by further ones is lost". In AIP-20 a second `transfer_private_to_commitment` into the same commitment therefore debits the payer again, leaves `total_supply` unchanged and credits nothing the recipient's wallet can find; the framework leaves the single-completion guarantee to contract logic (aztec-packages #14364), and the standard's token has not written it. CMTAT-Aztec has: `pay_commitment` pushes one nullifier, `H(commitment, DOM_SEP_CMTAT_COMMITMENT_PAID)`, next to the completion, so a second payment into the same commitment is a duplicate nullifier — an invalid transaction that never lands, and the payer's

funds never move. The same guard covers the commitment a sender opens for itself with `transfer_private_to_public_with_commitment`. What it costs and what it changes:

- **+52 gates** on `transfer_private_to_commitment` in every variant (93,011 → 93,063 base and Debt, 86,812 → 86,864 Light); no storage change, no ABI change, nothing published — the nullifier is a different hash of the commitment than the completion log tag and is unlinkable to it without the commitment.
- **The failure is an invalid transaction, not a named revert.** The payer's simulation passes — it does not check the nullifier tree for a nullifier it is about to create — and the node refuses the transaction; in the TXE this surfaces as `Nullifier collision`, the same signature as a replayed authwit. A wallet that wants to warn before sending can derive the nullifier and check its existence.
- **Behaviour differs from AIP-20's code in one direction only:** a caller reaching the shared selector meets a stricter contract, never a laxer one, and the wallet rule the standard's documentation gives (one commitment per expected payment) still holds; it is now enforced rather than assumed. Measured and tested in `tests/cmtat-aztec/src/test_edge_cases.nr`; the reasoning and the options considered are in [doc/technical/commitment-reuse.md](#).

Where the code lives. The entry points are in each variant's `main.nr` under the `HYBRID` banner (Noir requires every external function in the contract module); the state and the helpers, which are derived from the AIP-20 `Token`, are in [lib/src/modules/hybridModule.nr](#). That one file is **MIT-only**, with Wonderland's copyright notice for the derived parts — see [Intellectual property](#).

AIP-20 private profile

Seven entry points carry the exact names and parameter types of the AIP-20 `Token` in the [CMTA fork of aztec-standards](#), and therefore answer its selectors — a caller reaches an Aztec function by selector, which is derived from the name and the parameter *types* only:

Entry point	AIP-20 selector	Note
<code>transfer_private_to_private(from, to, amount, authwit_nonce)</code>	<code>0xedc09d49</code>	May also revert for compliance reasons; AIP-20's may too, through its hook
<code>mint_to_private(to, amount)</code>	<code>0xf8f84119</code>	<code>MINTER_ROLE</code> here, a single immutable minter there; identical from the caller's side
<code>name()</code> , <code>symbol()</code> , <code>decimals()</code>	<code>0x5c5c9c42</code> , <code>0x62cc9647</code> , <code>0x6bff8f59</code>	The <code>private_get_*</code> variants remain as this project's extras
<code>balance_of_private(owner)</code> , <code>total_supply()</code>	<code>0x4375727c</code> , <code>0x8dd382ec</code>	

`tests/cmtat-aztec/src/test_aip20_profile.nr` pins these values, read from the fork's compiled `Token::interface()`.

This is a **partial profile, not conformance**. Aztec has no interface detection, so the gaps show up at the first call rather than at discovery:

- `burn(account, amount, authwit_nonce)` deliberately keeps its own name and selector. AIP-20's `burn_private` is holder-authorized; CMTAT's burn is redemption, an issuer act gated by `BURNER_ROLE` on top of the holder's consent. A wallet calling `0xc282ed79` — the selector of that `burn_private`, which the standard defines as a holder-authorized self-burn — would call it as exactly that and fail with a role error it cannot anticipate.
- The four private/public bridges, `initialize_transfer_commitment` and `balance_of_public` are present since 0.4.0 but **behind the `public_side_enabled` deployment flag** — see [Private/public bridges](#). Still absent: `transfer_public_to_public`, `transfer_public_to_commitment`, `mint_to_public`, `mint_to_commitment`, `burn_public` and `get_auth_contract`.
- The constructor differs, so deployment tooling differs regardless.
- `transfer_batch`, `mint_batch`, `burn_batch` and `cancel_authwit` are this project's extras with no AIP-20 counterpart.

Deployment

Sandbox

Use these deployment instructions for quick testing.

Get the **sandbox, aztec-cli, and other tooling** with this command:

```
bash -i <(curl -s https://install.aztec.network)
```

Install the correct version of the toolkit with:

```
aztec-up install 5.2.0
```

The version should match the [Nargo.toml](#) dependency versions. More instructions [here](#)

Start the sandbox with:

```
aztec start --local-network
```

Run:

```
yarn install
yarn compile
yarn codegen
yarn test
```

The contract is deployed on the sandbox, by the [setup function](#), and all the tests are run.

Testnet

Use these deployment instructions for Testnet interactions. Testnet interactions are possible via scripts in the `./scripts` folder. With the below commands, we run the `deploy_contract.ts` script.

Run:

```
yarn compile
yarn codegen
yarn deploy
```

If you run into troubleshooting issues, consult the [Aztec starter repository](#) and try running it first.

Tests

Two suites, run together by `yarn test`.

- `yarn test:nr` runs the Noir tests under [tests/](#) — 239 across the five contracts, in the TXE, needing no network.
- `yarn test:js` runs the TypeScript end-to-end tests under [src/test/e2e/](#) — 17, against a local network (`aztec start --local-network`) and a `.env` supplying `L1_MNEMONIC` (`cp .env.example .env`). Set `SKIP_SANDBOX=true` when a network is already running, so the suite does not start its own.

A freshly deployed token is unusable for an hour of chain time, because every mint, transfer and burn reads `issuer_address` and that value is delayed — see [Delay of the delayed values](#). The end-to-end suite does not sit through it: it warps the chain past the delay, which is what keeps a full run to minutes rather than hours.

[doc/technical/test.md](#) describes what each end-to-end file covers, how the warp works and why a warp alone is not enough, and two defects the suite carried until 0.4.0 — a Fee Juice claim made one block before its L1-to-L2 message was available, and a hard-coded fee bound the protocol's fees had outgrown by a factor of 757. Read it before hard-coding a block count or a fee anywhere in the suite.

Gas sponsorship

What Fee Juice is. Every Aztec transaction pays for its own computation in **Fee Juice**, the protocol's native fee asset: bridged from Ethereum through the enshrined `FeeJuicePortal`, held as a *public* balance, and **non-transferable** — it can be spent on fees and on nothing else, and it cannot be sent from one account to another. Those two properties are what make the rest of this section matter: an issuer cannot simply top up its holders the way it would subsidise gas with an ERC-20, and a holder who pays its own fee publishes that it transacted, because a public balance visibly decreases.

A holder does not need Fee Juice to use this token, and the token carries no code to make that true. On Aztec the fee payer is chosen per transaction, not configured in the contract: any transaction may nominate a **fee-paying contract** (FPC) with `set_as_fee_payer()` during its non-reversible setup phase. There is no trusted forwarder, no `_msgSender()` override and no relayer to trust — which is why this implementation has no equivalent of CMTAT's ERC-2771

module and does not need one. The equivalency assessment answers the *fee payer / gasless* row `n.a.` for exactly this reason: the criterion asks for a module the protocol makes redundant.

Three ways to pay, all available to a holder of this token without any change to it:

Who pays	How	Where it works
The holder	Its own public Fee Juice balance, bridged from L1	Everywhere
A sponsored FPC	Pays unconditionally, asking nothing of the holder	Local network, devnet, testnet
A third-party FPC	Holds its own Fee Juice and charges the holder in another asset, usually against a signed quote and an authwit collected during setup	Anywhere one is deployed; the realistic mainnet route

This repository already uses the second: `src/utls/sponsored_fpc.ts` supplies the payment method, `deploy_account.ts` and `create_account_from_env.ts` deploy accounts with it, and `yarn fees` demonstrates all three paths including bridging and claiming Fee Juice.

An FPC cannot practically charge in this token. Two obstacles, and both are outside the contract's control: the setup phase runs against an allowlist from which custom token public functions were removed in Aztec 4.2.0, and mainnet alpha does not include custom token class IDs in it; and this token's balances are private notes, so a setup-phase charge would be a private transfer whose freeze and list screening could revert a phase that is not supposed to revert. An issuer wanting sponsorship should sponsor directly, or use an FPC that accepts a different asset.

Sponsorship is also a privacy measure, not only a convenience. Fee payment is public — a Fee Juice balance visibly decreases — so a holder who pays its own fee publishes that it transacted, even though the transfer itself reveals neither party nor amount. Paying through an FPC is what keeps the payer out of that record, which matters more for this token than for a transparent one.

Comparison with solidity CMTAT

What private CMTAT can do today

- **Mint/transfer:** Behave the same way as in CMTAT.
- **Burn:** We can perform `burn_from` with allowance.
- **Validation module:** Whitelisting and blacklisting are enabled on demand. The rule engine has been merged into the validation module, providing one interface that manages both and is always deployed along the main contract. The functionalities are private; storage can be read in public.
- **Pause module:** Same functionalities as CMTAT. Pause is public and instantaneous. Deactivation follows the CMTAT Solidity model: `deactivate_contract` requires the admin role and an existing pause, and once set it blocks `unpause_contract` forever, so the token can never move again. `public_get_deactivated` reads the flag.
- **Enforcement module:** Freeze and unfreeze are supported. Functionalities are private; storage can be read in public. There is a delay.
- **Access control module:** Same functionalities as CMTAT. Admin has the default role, which can be used to grant roles to themselves or others.

- **Version:** `version()` returns the implementation version as a compile-time constant, as CMTAT Solidity's `VersionModule` does. Aztec's contract class ID identifies the deployed artifact, but being a hash it neither orders releases nor matches a release tag, so the two are complementary.
- **Extra information module:** `set_token_id` / `token_id` carry the CMTAT token identifier, and `set_terms` / `terms` carry the reference to the legally required documentation, using the CMTAT Solidity notation (`DocumentInfo` of name, URI and document hash, with `lastModified` stamped by the contract). Guarded by `EXTRA_INFORMATION_ROLE`.
- **Credit events and debt modules:** Same functionalities as CMTAT. The debt record mirrors the Solidity `ICMTATDebt` interface field for field: a `DebtIdentifier` (issuer name and description, guarantor, debtholder representative) followed by a `DebtInstrument` (rate, par value, minimum denomination, dates, conventions, payment currency).

What will we be able to do in the future?

- **Larger batches:**
 - The cap is currently 4 addresses per call, set by the per-call note-hash and log budgets rather than by the private-call budget — see [Batching limits](#).
 - As those budgets grow, the cap can be raised: the logic is already written for arbitrary batch sizes. Each raise needs re-measuring rather than re-reading the constants, and the per-recipient proving cost grows with it.

These functions are not separated into their own “abstract contract”, which does not exist in Aztec. They are, since 0.4.0, in a library module: `lib/src/modules/tokenModule.nr` holds the value-moving chains once for the three variants, and the extra code this was expected to cost turned out to be negative — about 500 lines fewer across the repository, with every private circuit identical to the gate. What Noir still requires in each contract is the `#` `[external]` declarations, their attributes, the `enqueue_self` calls and the event emissions; see [doc/technical/token-module.md](#).

- **Validation module enhancements:**
 - The limitation regarding `DelayedPublicMutable` delay means changes to the whitelist/blacklist have a delay (minutes to hours) before reflecting on the blockchain.
 - A sanction-list mode is not provided, for lack of an on-chain list to check against — there is no Aztec equivalent of the [Chainalysis oracle](#) used on Ethereum.
- **Audit capabilities:**
 - Users may, in the future, be able to arbitrarily share to third-parties a shareable key for audit purposes.
- **Event management:** every state-changing entry point emits an event — see [Events](#). The follow-on this section used to carry as open is **done** since 0.4.0: mint and burn now deliver a constrained `Transfer` event to the issuer as well, naming the private party the public halves do not publish, so the issuer's event stream is a complete ledger of every movement. What is still open is the other direction — a *holder*-facing record of mint and burn, which today reaches the holder only as a note.

What will we never be able to do by design?

- **Force burning without consent:**

- We will never be able to burn someone else's tokens without their approval.

This could be possible if the token is implemented at the account contract level, and the issuer has shared nullifiers with the user for that specific account that holds notes for this token.

- **Immediate shared state changes:**

- We cannot have a shared state (public and private) that has no delay when changed, due to the protocol's construction.

Comparison with CMTAT-Confidential (Zama FHE)

[CMTAT-Confidential](#) is the other confidential CMTAT implementation: an [ERC-7984](#) Solidity token whose balances and amounts are encrypted with Fully Homomorphic Encryption on the [Zama protocol](#). It solves the same regulatory problem with a different cryptographic primitive, so the two make opposite trade-offs. Compared here against **v1.0.0** of that project.

The one-line difference: Aztec hides *who*; Zama FHE hides *how much*. On CMTAT-Confidential an observer still sees that address A transacted with address B and when — only the value is encrypted. Here, the counterparties and the transaction graph are private too, but the total supply is deliberately public.

Privacy technology

Axis	private-CMTAT-aztec	CMTAT-Confidential (Zama FHE)
Privacy primitive	Zero-knowledge proofs over a UTXO note model	Fully Homomorphic Encryption over a single encrypted balance
Where computation happens	Client side, in the user's PXE; the network verifies a proof	Symbolically onchain, with the real FHE computation offchain on Zama's coprocessor network
What is hidden	Amount, balance, sender, recipient and the transaction graph	Amount and balance only — addresses, counterparties and timing stay public
What stays public	<code>total_supply</code> , pause state, roles, freeze and list flags	Addresses and call graph, roles, pause and freeze state (supply encrypted by default)
Trust assumption for confidentiality	None beyond the protocol's cryptography	A threshold MPC key-management service holds the decryption key — no single party, but not nobody
Host chain	Aztec L2 only	Any EVM chain running the Zama protocol
Language and framework	Noir + aztec-nr v5.2.0	Solidity <code>^0.8.27</code> + <code>@fhevm/solidity</code> + OpenZeppelin Confidential Contracts

Axis	private-CMTAT-aztec	CMTAT-Confidential (Zama FHE)
Standards	None formal; a custom mapping of the CMTAT specification	ERC-7984, partial ERC-7943, ERC-1643 documents

Balances, supply and range

Axis	private-CMTAT-aztec	CMTAT-Confidential
Balance representation	A set of <code>Uints(u128)</code> , summed inside the owner's PXE	A single <code>uint64</code> handle
Maximum value	<code>u128</code> , about 3.4×10^{38}	<code>uint64</code> , about 1.84×10^{19} — decimals above 18 are rejected at construction
Reading a balance	Only the owner's PXE, plus the copy delivered to the issuer	Anyone holding an ACL grant, through the Zama relayer and threshold decryption
Total supply	Public by design and updated on every mint and burn	Encrypted by default ; opened either to registered observers or once-and-for-all with <code>publishTotalSupply</code>
Supply leakage	Accepted by design — each mint and burn amount is inferable from the public delta	Audit finding OZ-L-01: sequential disclosures leak individual mint and burn amounts; accepted as residual risk
Insufficient balance	Reverts (<code>Balance too low</code>)	Transfers zero silently via <code>FHESafeMath</code> , since reverting would leak the balance

Compliance and control

Feature	private-CMTAT-aztec	CMTAT-Confidential
Pause	✓ public, immediate	✓ immediate
Freeze an address	✓ but delayed by <code>CHANGE_ROLES_DELAY_SECONDS</code>	✓ immediate
Blacklist / whitelist	✓ both, delayed ; no sanction-list mode	✓ allowlist variant, immediate
RuleEngine / transfer hook	✗ (merged into the validation module)	✓ dedicated variant, though it passes <code>value = 0</code> because the amount is encrypted

Feature	private-CMTAT-aztec	CMTAT-Confidential
Forced transfer	✗ impossible by construction — the issuer cannot compute another holder's nullifiers; freezing is the workaround	✓ <code>forcedTransfer()</code>
Forced burn	✗ — burning needs the holder's authwit	✓ <code>forcedBurn()</code> , and it works on frozen addresses
Delegated spending	Authwit: single use, nonce-nullified, bound to one exact call	Operator system: time-limited authorisation
Partial token freeze	✗	✗
Snapshot	✗	✗
Upgradeability	✗	✗
Terms (name, URI, document hash)	✓ <code>set_terms</code> / <code>terms</code> , under <code>EXTRA_INFORMATION_ROLE</code> ; the <code>bytes32</code> hash is stored as two 128-bit halves, since a <code>Field</code> holds ~254 bits	✓
Token identifier	✓ <code>set_token_id</code> / <code>token_id</code> , same role	✓
Document registry (ERC-1643)	✗ — the terms are the one document; no named documents, no <code>getAllDocuments</code> , no removal	✓
Credit events and debt	✓	✗
Mutable name / symbol	✗ (<code>PublicImmutable</code>)	✓ post-deployment setters
Roles	11, numeric, in public state	14, named, OpenZeppelin <code>AccessControl</code>
Deployment variants	3 token variants (<code>CMTATAztec</code> , <code>CMTATAztecDebt</code> , <code>CMTATAztecLight</code>), plus 2 ARC-403 authorization contracts	4 (Lite, standard, RuleEngine, Whitelist)

Forced transfer is the sharpest divide, and the strongest argument for the FHE variant in a regulated deployment: CMTAT provides it for regulatory recovery — as an optional capability, criterion 22 of the equivalency assessment, not a mandatory one — it is a hard cryptographic impossibility here, and it is an ordinary function under FHE because the contract can compute on ciphertext it does not own.

Issuer auditability

Axis	private-CMTAT-aztec	CMTAT-Confidential
Mechanism	Every note is delivered twice — once to the owner, once to the issuer (<code>deliver_to</code>)	ACL grants to registered observers, re-granted automatically on every balance update
Granularity	Per note, so the issuer reconstructs the full history	The current balance handle, plus optional total-supply observers
Onchain guarantee	Partial. Every transfer's <code>Transfer</code> event reaches the issuer <code>onchain_constrained</code> — an on-chain, data-available record of sender, recipient and amount that the sender cannot forge. The <i>note</i> copies are still delivered offchain, because PXE cannot discover a note it does not own; see Issuer's view of transactions and notes	Onchain ACL, and a grant once made is irrevocable
Revocation	<code>set_issuer</code> , after the delay, stops future copies going to the old issuer; copies already delivered remain, since a delivered note cannot be recalled	Removing an observer stops future grants; past grants are irrevocable

Maturity

Axis	private-CMTAT-aztec	CMTAT-Confidential
Security audit	None — see the disclaimer at the top of this file	OpenZeppelin audit of v1.0.0
Audit scope caveat	—	The audit excluded the CMTAT library itself (pinned to an unaudited release candidate), the RuleEngine, the OpenZeppelin confidential contracts and the FHEVM

Axis	private-CMTAT-aztec	CMTAT-Confidential
Network status	Aztec mainnet ("Alpha") is live, with a Sepolia testnet; both ran 5.1.0 against this project's 5.2.0 when last checked, and the API still changes heavily between majors	Deployable on EVM mainnet wherever the Zama protocol is available
Batching	Capped at <code>MAX_ADDR_PER_CALL</code> by the per-call protocol limits	Ordinary Solidity loops, bounded only by gas
Fees	Fee juice or a sponsored FPC, plus client-side proving cost	Ordinary gas plus FHE compute units

Choosing between them

- Choose **Aztec** when the *relationship* is the secret — who holds what, and who traded with whom — and you can accept a public total supply and the absence of forced transfer.
- Choose **CMTAT-Confidential** when you need the full regulatory toolkit (forced transfer and burn, RuleEngine, documents), an audited codebase and deployment on an existing EVM chain, and it is acceptable that the transaction graph is public while amounts are not.

Note that the two disagree about total supply in opposite directions: this implementation publishes it deliberately, while the FHE implementation encrypts it and treats disclosure as a leak vector.

Limitations

- **Issuer's view of user balances:** [SEE](#)
- **Forced transfer:** [SEE](#)
 - Depending on the jurisdiction, a forced-transfer function may be required — for a court-ordered transfer, a lost-key recovery or an inheritance. This implementation cannot offer one.
 - **Current limitation:** This is not possible in Aztec as it would require the issuer to nullify a user's notes without consent.
 - **Workaround:**
 - Freeze the account.
 - If the account is frozen indefinitely, decrease the circulating supply. As a central issuer, I know the number of tokens the user has, so I can decrease supply accordingly.

Note: account freeze could reveal how much tokens a user had.

- **Rotating the issuer does not recall past copies:** [SEE](#)
 - `set_issuer` redirects *future* audit copies after the delay. Every note copy already delivered to the previous issuer stays with it — there is no mechanism, on any ledger, to un-deliver an encrypted message.
 - **Consequence:** a compromised issuer key keeps the audit history it already holds. Rotation limits the damage going forward; it does not undo it.

- **Consequence:** the new issuer's PXE must be live and registered from `effective_at` onwards, or copies sent during the gap are lost to the issuer side — they still reach the holders.
- **DelayedPublicMutable delay:** [SEE](#)
 - Note that, depending on the underlying ledger, a freeze may not be instantaneous: on a public blockchain the freeze transaction is visible in the mempool until it is included, and the target can pay to be ordered ahead of it, which a private relay such as Flashbots Protect avoids. Here the window is instead deterministic and protocol-enforced. See `doc/cmtat-assessment/cmtat_suggestion.md`.
 - Freezing and blacklisting addresses take effect only after a delay, measured in seconds, due to the `DelayedPublicMutable` type. Before Aztec v3 this delay was expressed in blocks.
 - **Options:**
 - Accept the delay.
 - Encrypt the blacklist with a key (implementation unclear).
- **Protocol limitations:**
 - A private function may emit only **16 private logs** and create only **16 note hashes** per call, which is what caps batching at 4 addresses — see [Batching limits](#).
 - The **8 nested private calls** per call limit does not affect batching here, because the batch helpers are inlined.

Miscellaneous

- **Wallet responsibilities:**
 - The wallet should implement note discovery and tagging mechanisms, not the application.
- **Mint function restrictions:**
 - Should we restrict the "to" address to not be the issuer to prevent a malicious issuer from hiding the real supply of the token by minting tokens to themselves?
- **Contract modification:**
 - Can a user modify a token contract function? No, it is not possible as each function is committed on the public state.
- **Encryption details:**
 - Encryption of note emission is done with AES-128. It's currently unclear if the encryption with AES is constrained at the protocol circuit level.
- **Transaction details:**
 - Notes are linked to their transaction hash because they are in the same transaction object when waiting in the mempool.
 - The transaction object cannot be modified between the point when it has been locally proven and when it reaches the sequencer because the output of the private kernel circuit is the input to the public kernel circuits, which it also verifies.
- **Replay attacks:**
 - The transaction hash is always emitted during local execution as the first nullifier of the transaction to prevent replay attacks. This is enforced by the private kernel circuit.
- **External references:**
 - Aztec Development Notes: [Engineering Designs](#)

- Protocol Limitations: [Aztec Protocol Circuits](#)

FAQ

Q: During the delay, can the target address know that it is about to be frozen?

Yes, from three public sources, so the delay is a notice period rather than a countdown the target cannot see:

- **The event.** `freeze` emits a public `AddressFrozen { account, is_frozen, enforcer, effective_at }` log in the same transaction, naming the address, who froze it and the exact timestamp the freeze takes effect.
- **The scheduled value.** A `DelayedPublicMutable` is public state, and so is its pending change: the library's own words are that the value "is fully public, as are all scheduled value and delay changes". Anyone with a node can read the scheduled flag and its effective timestamp before it is current.
- **The transaction.** The `freeze` call is a public function whose arguments, `user` among them, are visible in the block.

The consequence is the one this document already records for the validation module: a holder who sees the freeze scheduled can move its funds to a fresh address before it bites, and that address's flags are clean.

The delay does not create the problem, the public flag does. The delay only sets the length of the head start — one hour at the current `roles_delay()`, and whatever the admin last set it to afterwards.

Three responses, and only the first is available today:

- **Pause first.** `pause → freeze → wait the delay → unpause` reduces the head start to zero. It halts every holder for the delay, and the pause is itself a public event.
- **Check the freeze in the public half,** the way the pause is checked. Immediate, but it would publish the parties of every transfer — the trade this token refuses (review finding [H-6](#), option 5).
- **Remove the address from the event.** No help: the scheduled value and the call arguments stay public.

Q: Can the pause have a delay too, adjustable like the freeze delay but immediate by default?

Two things could be meant, and only one can exist.

- **A delay that would let the pause be read in private** (removing the public `_transfer` call): no. The storage type is fixed at compile time, so a `PublicMutable` cannot become a `DelayedPublicMutable` at runtime; and a `DelayedPublicMutable` with a delay of zero is not "immediate", it is unusable: a private read sets the transaction's expiry to `anchor + 0`, and the earliest including block is a slot later, so every transfer would be rejected. The floor is a few slots (the library's own example, 360 s, is "5 slots"), at which point the pause is no longer an emergency lever, which is the trade [H-3](#) refused.
- **A notice period before a pause takes effect,** default zero: possible and cheap. The flag stays a `PublicMutable` checked in public, a `pause_delay` setting and a `paused_at` timestamp are added, `pause_contract()` schedules `paused_at = now + pause_delay`, and the check becomes `now ≥ paused_at`. With the default of zero the behaviour is today's exactly. It is a planned-maintenance pause, not an emergency one, so it would come with an

emergency path that ignores the setting; it is not implemented, because nothing today needs a pause that announces itself.

Neither changes the freeze: the pause's immediacy is what closes a freeze's notice period (`pause` → `freeze` → `wait` → `unpause`), and a 3-second delayed pause would not shorten anything, it would make every transfer uncludable.

Q: Since every transfer calls the public pause check, does the one-hour delay still buy any privacy?

No, not for this token. The README records it as a correction to the way the delay was first argued.

The privacy effect of a delay is that the transaction's public expiry, `anchor + delay`, narrows the transaction to the contracts using that delay. But every value-moving operation here already enqueues a public call whose target is this contract:

- `_transfer()` for a transfer;
- `_mint` / `_burn`, with the role-holder and the amount;
- `_credit_public` / `_debit_public` for the bridges;
- the `is_burn` call of the authorization hook.

An observer therefore knows "this token, this operation" from that call before reading any expiry, so the expiry adds nothing about *which* contract, whatever the delay is set to. What it does not reveal is *who*: the enqueued call is a self-call, so its sender is the token, and the holder stays private either way.

Two jobs remain, and they are why the delay has a value at all:

- **The validity window.** It bounds the time a transaction has to be proved and included.
- **Enforcement latency.** It is how long a freeze, a listing, a list-mode change or an issuer rotation takes to bite.

At any value it also leaks the anchor block's timestamp, `expiry - delay`, which is a timing fact rather than an identifying one.

The privacy set would matter again only for a path with no public half. For transfers that would mean a delayed pause; mints and burns can never qualify, since the role check and `total_supply` are public state. See *What each operation publishes* above.

Q: Would burning be simpler with AIP-20's `burn_private` than with this token's `burn`, for an issuer that holds the tokens itself?

No, and the premise does not hold: AIP-20 uses the identical authentication-witness mechanism. Its `burn_private` carries `#[authorize_once("from", "_nonce")]`, the same macro and the same `from`-plus-nonce convention as this token's `burn`. The two differ in *authorisation* — holder-`authorised` there, `BURNER_ROLE` here — and not in the machinery around it.

For a party burning its own tokens, that machinery costs nothing in either contract. The macro validates a witness only when `msg_sender()` differs from the named account; the account itself passes `authwit_nonce = 0` and no witness is created, collected or nullified. An issuer burning its own treasury calls `burn(issuer, amount, 0)` and is done.

Burning one's own tokens	AIP-20 <code>burn_private</code>	this token's <code>burn</code>
--------------------------	-------------------------------------	--------------------------------

Burning one's own tokens	AIP-20 <code>burn_private</code>	this token's <code>burn</code>
Witness to create	none	none
Nonce	0	0
Additional requirement	—	<code>BURNER_ROLE</code> on the caller, granted once

If the comparison is against the ARC-403 hook rather than the authwit, it inverts. AIP-20's `burn_private` calls `_call_auth_private(from, amount)`, so a fork token wired to `CMTATAztecAuth` makes a cross-contract call to the authorization contract on every burn — `authorize_private` measures 14,650 gates — on top of the token's own work, and adds a second deployed contract whose pointer is immutable. This token screens freeze and lists inline, in the same circuit. For the issuer's own burn the standard-plus-hook route has more moving parts, not fewer.

One asymmetry does favour AIP-20. This token screens the **account** of a burn, and when the issuer burns its own tokens the account is the issuer, so an issuer that were itself frozen or outside an active whitelist could not burn its own treasury. A plain AIP-20 token with no hook applies no such check.

Whether that is a defect or the intended behaviour is a policy question: an issuer subject to its own enforcement role is defensible. It is a foot-gun either way, and worth deciding deliberately rather than discovering.

Glossary

Terms you need in order to read this repository. The first table is Aztec the protocol, the second is the Aztec.nr code — the modules in `lib/` and the contract variants in `contracts/` — and the third is CMTAT and the decisions specific to this project.

Aztec protocol

Term	Definition
Aztec	A privacy-focused Layer 2 on Ethereum. Every transaction has a private part, proven on the user's own device with zero-knowledge proofs, and a public part, executed by the network like an EVM transaction.
L1 / L2	L1 is Ethereum, where Aztec settles and where fee juice is bridged from. L2 is Aztec itself.
Private execution	Contract code run locally by the user. Its inputs and outputs stay secret; the network only sees a proof plus the note hashes and nullifiers it produced.
Public execution	Contract code run by the sequencer over public state, visible to everyone. Public calls made from private code run <i>after</i> all private execution, so they cannot return a value to it.

Term	Definition
Utility function	An unconstrained, offchain query (<code>#[external("utility")]</code>). It never appears in a transaction and carries no correctness guarantee — it is the Aztec analogue of an <code>eth_call</code> -only view. <code>balance_of_private</code> is one.
Note	The unit of private state: a small struct (here a <code>UintNote</code> holding a <code>u128</code> amount) whose <i>hash</i> is published onchain while its content stays private. A private balance is the sum of the notes a holder owns.
Note hash tree	The append-only onchain tree of note hashes. Append-only so that spending a note cannot be linked to its creation.
Nullifier	A deterministic, secret-derived value published when a note is spent. The protocol rejects duplicates, which is what prevents double-spending. Only the note's owner can compute it.
Nullifier tree	The append-only onchain tree of nullifiers. A note is unspent exactly when its nullifier is absent.
Note discovery	How a recipient learns that a note was created for them: the sender encrypts a message with the note's content and delivers it, and the recipient's PXE decrypts it and verifies the note hash onchain.
PXE	<i>Private eXecution Environment</i> — the client-side component holding a user's keys and private notes, and running private functions. Each user has their own; one PXE cannot read another's notes.
Sequencer	The network actor that orders transactions and executes their public parts.
AVM	The public virtual machine the sequencer runs, comparable in model to the EVM.
Authwit	<i>Authentication witness</i> — a signed authorisation letting a third party perform one specific action on your behalf. The Aztec equivalent of an ERC-20 <code>approve</code> + <code>transferFrom</code> , but scoped to an exact call and consumed once.
Fee juice	The native token used to pay transaction fees, bridged from L1.
FPC / Sponsored FPC	<i>Fee Payment Contract</i> — pays fees on a user's behalf. The sponsored FPC pays unconditionally, which is how fresh accounts in this repo transact without being funded first.
Local network	A local Aztec network for development, started with <code>aztec start --local-network</code> . Older material calls it the <i>sandbox</i> , and <code>--sandbox</code> was the flag until Aztec 3.0; it no longer exists.
Testnet	The public Aztec test network, targeted by the scripts in <code>scripts/</code> .

Aztec.nr and the code in this repository

Term	Definition
Noir	The language Aztec contracts are written in. Rust-like syntax, but it compiles to zero-knowledge circuits, which is why there is no inheritance and no early <code>return</code> .
Aztec.nr	The Noir framework providing the contract macros, state variables and note types. Pinned to v5.2.0 here.
TXE	<i>Test eXecution Environment</i> — the harness behind <code>aztec test</code> that runs Noir tests against a simulated network. Everything under <code>tests/*/src/</code> targets it.
<code>#[external("private" \ "public" \ "utility")]</code>	Marks a function callable from outside the contract, and says which environment runs it.
<code>#[internal("private" \ "public")]</code>	A helper callable only from inside the contract and inlined at the call site — reached through <code>self.internal</code> . <code>_grant_role_internal</code> , <code>_emit_listed</code> and <code>_open_commitment</code> are these; the value-moving chains are ordinary library functions in <code>tokenModule.nr</code> , inlined the same way.
<code>#[only_self]</code>	A real (non-inlined) function only the contract itself may call. The enqueued public halves <code>_mint</code> , <code>_transfer</code> and <code>_burn</code> use it.
<code>self.enqueue_self</code>	Schedules one of this contract's public functions to run after private execution. This is how a private mint updates the public <code>total_supply</code> . Not a Noir feature: the <code>#[aztec]</code> macro generates one method per non-view public function of <i>this</i> contract, so <code>self.enqueue_self._mint(...)</code> is type-checked against the real signature and a rename breaks the call site. <code>#[view]</code> functions go to <code>self.enqueue_self_static</code> instead, and the generated stub publishes <code>msg_sender</code> — <code>self.enqueue_incognito</code> is the variant that does not.
<code>#[authorize_once("from", "authwit_nonce")]</code>	Macro that validates the authwit when the caller is not <code>from</code> , and nullifies the nonce so it cannot be replayed. The <code>from</code> account itself must pass <code>authwit_nonce = 0</code> .
Storage slot	The index that keeps one state variable's data from colliding with another's. Assigned automatically.
<code>PublicMutable<T></code>	Public value, read and written by public functions only. Used for <code>total_supply</code> and the role table.
<code>PublicImmutable<T></code>	Public value written once and readable everywhere, including private functions. Used for <code>name</code> , <code>symbol</code> , <code>decimals</code> .

Term	Definition
<code>DelayedPublicMutable<T, DELAY></code>	A public value whose writes take effect only after <code>DELAY</code> . That delay is what makes it readable from a <i>private</i> function, since the circuit can prove the value cannot change for a known window. Used for <code>issuer_address</code> , freeze flags and validation flags. DELAY is a number of seconds (<code>CHANGE_ROLES_DELAY_SECONDS = 3600</code> , one hour, and adjustable at runtime with <code>set_roles_delay</code>), not a block count.
<code>Owned<V></code>	Wrapper required by private state variables, binding them to an owner; reached with <code>.at(address)</code> .
<code>PrivateSet<Note></code>	A collection of notes belonging to one owner.
<code>BalanceSet</code>	The Aztec.nr state variable for private balances, a <code>PrivateSet<UintNote></code> with <code>add / sub / balance_of</code> . <code>private_balances</code> is an <code>Owned<BalanceSet></code> .
<code>Map<K, V></code>	Key-value container for <i>public</i> state, the analogue of a Solidity <code>mapping</code> . Private state uses <code>Owned</code> instead.
<code>UintNote</code>	The built-in note type holding a <code>u128</code> , used here for token amounts.
Note message / <code>MessageDelivery</code>	Creating a note yields a message that must be delivered, and you choose how: <code>onchain_constrained()</code> (proven, most expensive), <code>onchain_unconstrained()</code> (onchain but trusts the sender), or <code>offchain()</code> (cheapest, no onchain data). See <i>Issuer's view of transactions and notes</i> for the choice made here.
<code>deliver_to(address, mode)</code>	Delivers a copy of a note or event message to somebody who is <i>not</i> the note's owner. They learn the note exists; they cannot spend it, and cannot see when it is spent. This is the issuer's audit channel. An Aztec.nr method, not a Noir one, on the message that <code>self.emit(...)</code> and <code>BalanceSet::add / sub</code> return; the message is <code>#[must_use]</code> , so forgetting to deliver it is a compiler warning rather than silent data loss. <code>deliver(mode)</code> is the shorthand for the owner. A note copy to a non-owner has to be <code>offchain()</code> — a stock PXE cannot process one delivered onchain, because it cannot compute the nullifier; an event copy has no owner and can be <code>onchain_constrained()</code> .

Term	Definition
Module (in this repo)	Because Noir has no inheritance, each concern is a plain struct held as a field of the contract's storage: <code>access_control</code>, <code>pause_module</code>, <code>enforcement_module</code>, <code>validation_module</code>, <code>extra_information_module</code>, and in the Debt variant <code>credit_event_module</code> and <code>debt_module</code>. They live in <code>lib/</code>, shared by every variant, and each user-callable entry point is still re-declared in that variant's <code>main.nr</code>.
<code>EmbeddedWallet</code>	The TypeScript wallet used by <code>scripts/</code> and the end-to-end tests. It owns its own PXE and holds several accounts; each call names its sender with <code>from</code>.
<code>aztec codegen</code>	Generates the typed TypeScript contract bindings in <code>src/artifacts/</code> from the compiled artifact. Re-run it after any change to the contract's interface.

CMTAT and this project

Term	Definition
CMTAT	<i>Capital Markets and Technology Association Token</i> — a standard for tokenising securities in line with local regulation. This repository is a private implementation of it.
Security token	A token representing a regulated financial instrument (a bond, an equity share, a private credit note) rather than a utility asset.
Issuer	The institution that issues the token. It mints and burns, and it receives a copy of every note so it can audit holdings. Its address lives in <code>issuer_address</code>.
Admin	Holder of <code>DEFAULT_ADMIN_ROLE</code> (role <code>1</code>), the only role that can grant and revoke the others. Granted at deployment. Note that <code>getRoleAdmin</code> returns <code>DEFAULT_ADMIN_ROLE</code> for every role, including itself, so an admin can appoint another admin, revoke one, or step down with <code>renounce_role</code>. An admin cannot revoke itself; that is what <code>renounce_role</code> is for.

Term	Definition
Role	A numeric permission checked in public state: <code>DEFAULT_ADMIN_ROLE</code> 1, <code>PAUSE_ROLE</code> 2, <code>ENFORCEMENT_ROLE</code> 3, <code>VALIDATION_ROLE</code> 4, <code>ADDRESS_LIST_ADD_ROLE</code> 5, <code>ADDRESS_LIST_REMOVE_ROLE</code> 6, <code>MINTER_ROLE</code> 7, <code>BURNER_ROLE</code> 8, <code>DEBT_ROLE</code> 9, <code>DEBT_CREDIT_EVENT_ROLE</code> 10, <code>EXTRA_INFORMATION_ROLE</code> 11.
Authorisation module	The role table (<code>access_control</code>) plus <code>only_role</code> , the check every other module calls.
Pause module	A public on/off switch. While paused, transfers revert, because <code>transfer_private_to_private</code> enqueues a public call that asserts the contract is not paused. Mint and burn continue through a pause, as in CMTAT Solidity, and stop only at deactivation.
Enforcement module	Per-address freezing. A frozen address can neither send nor receive. Because the flag is a <code>DelayedPublicMutable</code> , a freeze takes effect only after the delay.
Validation module	Transfer restriction by address list. Holds each address's flags and the switch saying which lists are enforced.
Blacklist / whitelist	The two list modes (<code>BLACKLIST_FLAG</code> 1, <code>WHITELIST_FLAG</code> 2). Blacklist blocks listed addresses, whitelist allows only listed ones. Exactly one mode is enforced per transfer.
Credit events extension	CMTAT bond attributes recording default, redemption and rating.
Debt extension	CMTAT bond attributes, mirroring the Solidity <code>ICMTATDebt</code> : a <i>debt identifier</i> (issuer name and description, guarantor, debtholder representative) and a <i>debt instrument</i> (interest rate, par value, minimum denomination, issuance and maturity dates, coupon frequency, interest schedule and payment date, day-count and business-day conventions, payment currency and its contract address).
Total supply	Deliberately public . Balances are private, but the number of tokens in circulation is not, and it moves visibly on every mint and burn.

Term	Definition
Force transfer	The CMTAT power to move a holder's tokens without their consent. Not possible here , because the issuer cannot compute another holder's nullifiers. Freezing the account is the workaround — see <i>Limitations</i> .
Batch functions	<code>mint_batch</code> and <code>burn_batch</code> , capped by <code>MAX_ADDR_PER_CALL</code> (currently <code>4</code>), and <code>transfer_batch</code> , capped by <code>MAX_TRANSFER_ADDR_PER_CALL</code> (currently <code>2</code>) because each recipient costs four constrained deliveries. Both caps are measured, not derived — see Batching limits .
<code>CHANGE_ROLES_DELAY_SECONDS</code>	The <i>initial</i> delay, in seconds (<code>3600</code> , one hour), before a scheduled change to a freeze flag, a list entry, the operations switch or the issuer address becomes current; the admin can change it at runtime with <code>set_roles_delay</code> (see <i>Delay of the delayed values</i>). Nothing that reads those values sees the new one before it elapses. It also gates the issuer address — set by the constructor, which is why no mint, transfer or burn works until the delay has passed after deployment, and rescheduled by <code>set_issuer</code> .

Intellectual property

The code is copyright (c) Capital Market and Technology Association, 2026, and is released under the [Mozilla Public License 2.0](#) and the [MIT license](#). You may choose either license.

Third-party code. `lib/src/modules/hybridModule.nr` contains code derived from the AIP-20 Token of [aztec-standards](#), Copyright (c) 2024 Wonderland, MIT License. That file is released under the MIT license only; its header carries the notice.

The history up to and including commit [61f4220d5565840fd4fcdd2b723c9f55eb824c60](#) (the 0.2.0 release, and so the 0.1.0, 0.1.1 and 0.2.0 releases) is copyright (c) 2025 Taurus SA, under the same two licenses. Later commits are copyright CMTA.

We are not aware of any patent or patent application covering the techniques implemented.

Security policy

Please see [SECURITY.md](#).